

Apostila – Banco de Dados II

José Corrêa Viana

jcorrea@unipam.edu.br

jcorreavian@hotmail.com

twitter.com/rhuodox

facebook.com/jcorreaviana

Patos de Minas, 2012

Agradecimentos

Gostaria de agradecer a todas as pessoas que confiam na minha pessoa e no meu trabalho. Sem elas, chegar até aqui seria impossível.

Agradeço em especial a minha família. Ao meu pai José Donisete, minha mãe, Gessi Aparecida e meus irmãos, Pedro Corrêa e Maria Laura Corrêa que sempre me apoiaram em todas as minhas decisões.

Agradeço separadamente a minha namorada Jessyka Cristina, que também faz parte da minha família, que me motiva e me encanta a cada minuto que se passa de minha vida.

Agradeço a todos os meus colegas de trabalho, professores, colegas da área de TI, pelo auxílio no desenvolvimento desse material e na ajuda de todos os dias. Em especial ao Fernando Corrêa por acreditar sempre em mim e ao Rafael Igor pelo esclarecimento de dúvidas sobre assuntos novos para mim.

Agradeço a todas as pessoas e empresas citadas nas referências, pelos valiosos repositórios para buscas.

Agradeço aos meus alunos que se depositaram em mim a possibilidade de aprender e ensinar.

E fechando com chave de ouro, agradeço a você que está lendo esse material agora.

Muito Obrigado!

O que você encontrará aqui

Um guia para esclarecimento, aprendizagem e atualização de informações sobre Banco de Dados, desde sua modelagem, aplicação de técnicas de melhoria de performance e ferramentas de gestão, tanto para a Equipe de TI, quanto para a equipe estratégica das Empresas.

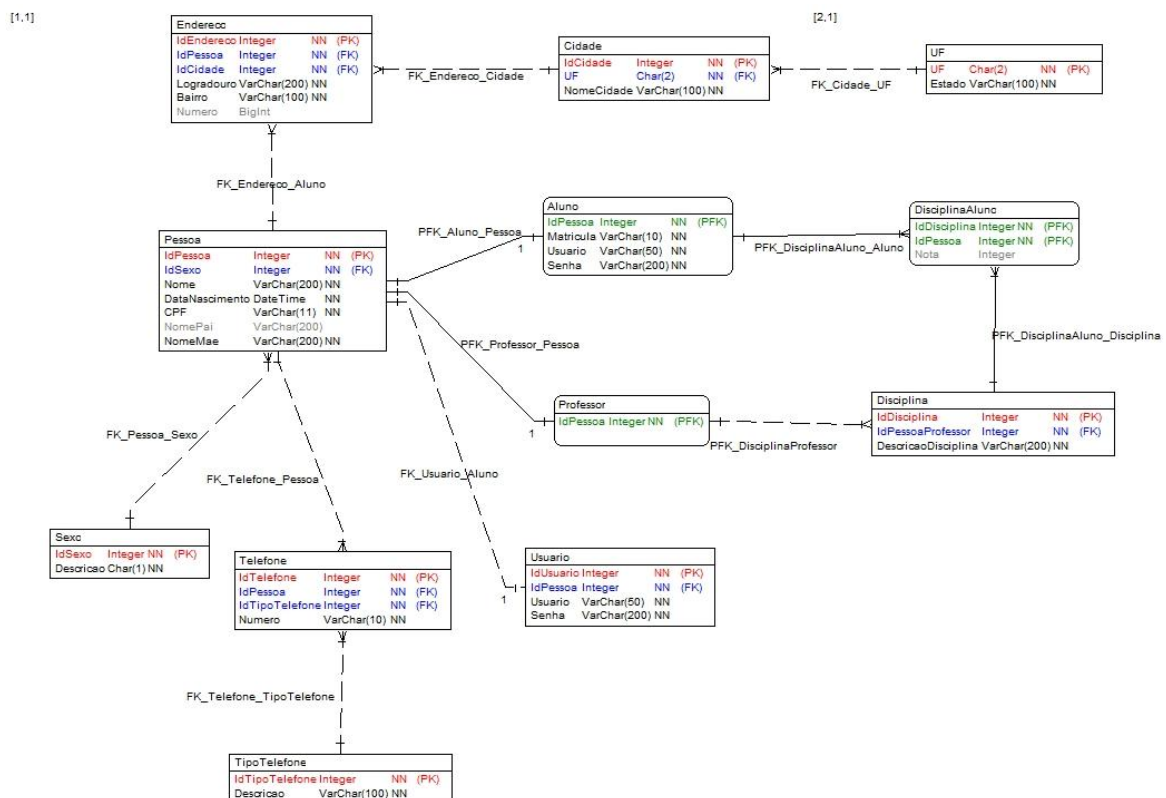
Dúvidas, sugestões e tudo mais que possa agregar valor à essa apostila, basta entrar em contato.

Primeiros Conceitos

Definindo basicamente a linguagem SQL, vimos que ela visa definir uma linguagem global de manipulação de dados, que teve início em 1970. Classificamos suas operações em três tipos, apresentados na tabela abaixo:

DML (Data Manipulation Language)	DDL (Data Definition Language)	DCL (Data Control Language)
• SELECT; • INSERT; • UPDATE; • DELETE.	• CREATE; • ALTER; • DROP.	• GRANT; • REVOKE.
Nível de dados	Nível de tabela	Nível de segurança

Na primeira aula realizamos um exercício para normalização de uma tabela, que representava um banco. Com a aplicação das três formas de normalização de banco, chegamos ao seguinte DER, seguindo algumas regras de negócio:



Lembrando que esse é a primeira versão do DER, que provavelmente terá modificações de acordo com novas regras de negócio definidas.

Após criarmos o DER, devemos criar um banco de dados, e, dentro desse banco, criarmos as tabelas, seguindo o modelo projetado:

Criando o Banco de Dados

- Para criarmos o banco de dados, usamos a seguinte sintaxe:

```
CREATE DATABASE Academico
```

- Ou seja, estamos criando um novo banco de dados, seguindo as configurações padrões do SQL Server 2008 para um novo banco de dados denominado “Academico”.

Criando as tabelas

Nesse primeiro instante, não iremos nos preocupar com os relacionamentos entre as tabelas. O script para geração das tabelas, de acordo com o DER, segue abaixo:

```
USE Academico
```

```
Create table [Pessoa]
(
    [IdPessoa] Integer Identity(1,1) NOT NULL,
    [IdSexo] Integer NOT NULL,
    [Nome] Varchar(200) NOT NULL,
    [DataNascimento] Datetime NOT NULL,
    [CPF] Varchar(11) NOT NULL,
    [NomePai] Varchar(200) NULL,
    [NomeMae] Varchar(200) NOT NULL,
    Primary Key ([IdPessoa])
)
go

Create table [Disciplina]
(
    [IdDisciplina] Integer Identity(1,1) NOT NULL,
    [IdPessoaProfessor] Integer NOT NULL,
    [DescricaoDisciplina] Varchar(200) NOT NULL,
    Primary Key ([IdDisciplina])
)
go

Create table [DisciplinaAluno]
(
    [IdDisciplina] Integer NOT NULL,
    [IdPessoa] Integer NOT NULL,
    [Nota] Integer NULL,
```

```

Primary Key ([IdDisciplina],[IdPessoa])
)
go

Create table [Endereco]
(
    [IdEndereco] Integer Identity(1,1) NOT NULL,
    [IdPessoa] Integer NOT NULL,
    [IdCidade] Integer NOT NULL,
    [Logradouro] Varchar(200) NOT NULL,
    [Bairro] Varchar(100) NOT NULL,
    [Numero] Bigint NULL,
Primary Key ([IdEndereco])
)
go

Create table [Cidade]
(
    [IdCidade] Integer Identity(1,1) NOT NULL,
    [IdUF] Integer NOT NULL,
    [NomeCidade] Varchar(100) NOT NULL,
Primary Key ([IdCidade])
)
go

Create table [UF]
(
    [IdUF] Integer Identity(1,1) NOT NULL,
    [UF] Char(2) NOT NULL,
    [Estado] Varchar(100) NOT NULL,
Primary Key ([IdUF])
)
go

Create table [Usuario]
(
    [IdUsuario] Integer Identity(1,1) NOT NULL,
    [IdPessoa] Integer NOT NULL,
    [Usuario] Varchar(50) NOT NULL,
    [Senha] Varchar(200) NOT NULL,
Primary Key ([IdUsuario])
)
go

Create table [Aluno]
(
    [IdPessoa] Integer NOT NULL,
    [Matricula] Varchar(10) NOT NULL,
    [Usuario] Varchar(50) NOT NULL,
    [Senha] Varchar(200) NOT NULL,
Primary Key ([IdPessoa])
)
go

Create table [Telefone]
(
    [IdTelefone] Integer Identity(1,1) NOT NULL,
    [IdPessoa] Integer NOT NULL,
    [IdTipoTelefone] Integer NOT NULL,
    [Numero] Varchar(10) NOT NULL,
Primary Key ([IdTelefone])
)

```

```

)
go

Create table [TipoTelefone]
(
    [IdTipoTelefone] Integer Identity(1,1) NOT NULL,
    [Descricao] Varchar(100) NOT NULL,
Primary Key ([IdTipoTelefone])
)
go

Create table [Sexo]
(
    [IdSexo] Integer Identity(1,1) NOT NULL,
    [Descricao] Char(1) NOT NULL,
Primary Key ([IdSexo])
)
go

Create table [Professor]
(
    [IdPessoa] Integer NOT NULL,
Primary Key ([IdPessoa])
)
go

```

- O comando `USE Academico` define em qual banco será executado o script, nesse caso, no banco “Academico”.
- Quando utilizamos o `IDENTITY(1,1)` estamos querendo dizer que o campo definido com essa cláusula será incrementado de um valor inicial (primeiro parâmetro) de acordo com o incremento definido (segundo parâmetro). Portanto, no nosso projeto, o campo terá valores 1, 2, 3..., n. Se fosse `IDENTITY(1,2)` seria 1, 3, 5, ..., n.
- `PRIMARY KEY ([NomeCampo])` É um exemplo de definição de chave primária em uma tabela. Outra maneira de definir um campo como chave primária poderia ser também:
 - `[NomeCampo] INTEGER IDENTITY(1,1) PRIMARY KEY NOT NULL`
- Com isso teremos o nosso banco de dados já com as tabelas criadas. Já podemos também inserir alguns dados no banco.

Comando Insert

Inicialmente, vamos fazer inserção em duas tabelas na respectiva ordem: UF e Cidade.

```
INSERT INTO UF(UF, Estado)
VALUES
('MG', 'Minas Gerais'),
('SP', 'São Paulo'),
('RJ', 'Rio de Janeiro')
GO

INSERT INTO Cidade (IdUF, NomeCidade)
VALUES
(1, 'Patos de Minas'),
(1, 'Uberlândia'),
(2, 'São Paulo'),
(3, 'Rio de Janeiro'),
(1, 'Lagoa Formosa')
GO
```

A relação entre um estado e uma cidade é o campo que define de qual estado pertence uma cidade.

```
CREATE TABLE [Cidade]
(
    [IdCidade] INTEGER IDENTITY(1,1) NOT NULL,
    [IdUF] INTEGER NOT NULL,
    [NomeCidade] VARCHAR(100) NOT NULL,
    PRIMARY KEY ([IdCidade])
)
GO
```

Nesse caso IdUF é o campo que faz essa ligação. Realizando algumas consultas:

```
SELECT * FROM UF
SELECT * FROM Cidade
```

- As consultas acima trazem todas as colunas e todos os registros da tabela.
- Já as consultas abaixo utilizam parâmetros de consulta.
 - A primeira consulta irá retornar todas as cidades que contenham “MINAS” no final de seu nome e o “%” indica que não há restrição para qualquer outro nome antes de “MINAS”.
 - A segunda retornará as cidades que tenham “LAGOA ” como nome inicial, ignorando qualquer restrição de consulta para o restante.

- Lembrando que o espaço antes da expressão também pode afetar a consulta, pois assim estamos dizendo que existe uma diferenciação caso exista nomes compostos, como no exemplo.

```
SELECT * FROM Cidade WHERE NomeCidade LIKE '% MINAS'
SELECT * FROM Cidade WHERE NomeCidade LIKE 'LAGOA %'
```

Comando Update

Através desse comando é possível realizar alterações nos dados de uma tabela. A inserção pode ser feita em uma ou mais linhas, de acordo com a necessidade. Vamos a alguns exemplos:

```
UPDATE Cidade SET NomeCidade = 'Americana' where IdUF = 2
```

- A instrução Update acima atualiza a tabela chamada “Cidade”, alterando o campo “NomeCidade” para “Americana” onde o “IdUF” seja igual a 2. Notamos que independente que quantidade de cidades que temos, TODAS as cidades que tenham “IdUF” = 2 terão como no campo “NomeCidade” o valor “Americana”.
- Porém podemos ser mais específicos e alterar o nome de apenas um registro por exemplo, através do seguinte comando:

```
UPDATE Cidade SET NomeCidade = 'São Paulo' where NomeCidade = 'Americana'
```

Com isso estaremos atualizando o valor para “São Paulo” somente onde campo “NomeCidade” possuir o valor “Americana”.

Alterando a estrutura de uma tabela

Existem DDL's para controle da tabela. Vamos falar da instrução “Alter Table”.

- Essa instrução pode ser utilizada para:
 - Adicionar uma nova coluna;
 - Alterar o tipo de dados de uma colua ou;
 - Remover colunas de uma tabela.

Vamos realizar uma alteração na tabela “Endereco”. Nessa tabela iremos adicionar o CEP do Endereco. Então vamos lá:

```
ALTER TABLE Endereco
ADD CEP DATETIME NULL
```

- Com essa estrutura estamos dizendo que vamos realizar uma alteração no banco, adicionando o CEP na tabela “Endereco”.
- Vamos realizar uma alteração nesse campo, pois sabemos que valor do tipo DateTime não é interessante (e nem é possível) para armazenar a estrutura do CEP de um endereço.

```
ALTER TABLE Endereco
ALTER COLUMN CEP VARCHAR (8) NULL
```

- Com essa estrutura estamos dizendo que vamos realizar uma alteração no banco, alterando a estrutura do campo CEP na tabela “Endereco”. Assim estamos determinando que o campo CEP agora será no tipo Varchar(8), que é o tamanho necessário para armazenar o dado de CEP.
- E se tivermos uma coluna que queremos excluir?
- Fácil! 😊 Iremos proceder da seguinte maneira (para apresentar essa cláusula iremos criar um campo somente para apresentarmos a estrutura para exclusão do campo):

```
ALTER TABLE Endereco
ADD Complemento2 VARCHAR(100) NULL
```

```
ALTER TABLE Endereco
DROP COLUMN Complemento2
```

- Com essa estrutura estamos dizendo que vamos realizar uma alteração no banco, excluindo o campo Complemento na tabela “Endereco”.
- Caso queiramos excluir mais de uma coluna também é possível, seria assim:

```
ALTER TABLE Endereco
DROP COLUMN Complemento2, Complemento3
```

Agora vamos fazer uma exclusão na tabela cidade, seguindo a expressão:

```
DELETE FROM Cidade WHERE NomeCidade = 'Lagoa Formosa'
```

- A consulta irá excluir o registro na tabela “Cidade” onde o nome seja igual à “Lagoa Formosa”;
- Como não está sendo utilizado a condição LIKE juntamente com “%”, a exclusão será efetuada somente para o registro que contenha exatamente o mesmo valor da expressão;
- Após a execução dessa expressão DML, a Cidade “Lagoa Formosa” será excluída do banco de dados.

Agora vamos excluir um estado, seguindo a expressão:

```
DELETE FROM UF where IdUF = 1
```

- Essa expressão DML acarretará na exclusão do estado onde o “IdUF” seja igual a 1.
- Diferentes condições podem ser utilizadas para a seleção dos valores a serem excluídos, como em uma consulta.

Porém, uma ação dessa traz alguns “efeitos colaterais” para as informações contidas em nosso banco de dados. Vamos pensar:

- Existia um estado chamado “Minas Gerais”;
- Existe cidades vinculadas a esse estado;
- Eu excluí o estado;
- E agora? ☹

Para evitar que ações como essas ocorram, existem restrições que podemos inserir no banco de dados, ajudando a manter a integridade das informações e garantindo que a regra de dependência entre as tabelas seja cumprida.

- Vamos começar fazendo essa restrição para o nosso relacionamento entre a tabela “UF” e a tabela “Cidade”;
- Sabemos que um estado é composto por uma ou mais cidades e que as cidades são pertencentes de um estado;

- Portanto vamos criar essa estrutura de dependência entre as informações:

```
ALTER TABLE Cidade
ADD CONSTRAINT FK_Cidade_UF FOREIGN KEY (IdUF) REFERENCES UF (IdUF)
```

- Uma mensagem de erro semelhante a mensagem abaixo irá surgir:

```
Msg 547, Level 16, State 0, Line 1
The ALTER TABLE statement conflicted with the FOREIGN KEY constraint "FK_Cidade_UF". The
conflict occurred in database "Academico", table "dbo.UF", column 'IdUF'.
```

- Isso acontece pois ainda existem registros na tabela “Cidade” que não possuem o campo que cria a dependência entre a tabela “UF”.
- Para criarmos a regra de integridade, vamos excluir os dados que referenciam um estado que não existe mais:

```
DELETE FROM Cidade WHERE IdUF = 1
```

- Com essa consulta, excluimos todas as cidades que possuíam “Minas Gerais” como estado.
- Vamos executar novamente a função DDL para criar a regra de integridade entre as tabelas:

```
ALTER TABLE Cidade
ADD CONSTRAINT FK_Cidade_UF FOREIGN KEY (IdUF) REFERENCES UF (IdUF)
```

- Agora sim notamos que o script foi executado com sucesso. Vamos tentar excluir um estado que possui cidades vinculadas a ele:

```
DELETE FROM UF WHERE IdUF = 2
```

- A seguinte mensagem de erro é exibida:

```
Msg 547, Level 16, State 0, Line 1
The DELETE statement conflicted with the REFERENCE constraint "FK_Cidade_UF". The
conflict occurred in database "Academico", table "dbo.Cidade", column 'IdUF'.
The statement has been terminated.
```

- Notamos que não foi possível realizar a exclusão do estado que possui o “IdUF” igual a 2.
- Isso ocorre pois existe no mínimo um registro que referencia o valor que queremos excluir, no caso, a cidade de “São Paulo”.
- Existem outras maneiras de se criar a regra de integridade.

- DICA: É mais cômodo realizar a criação das Constraints após ter criado todas as tabelas, pois, para existir um relacionamento entre duas tabelas, é necessário que as mesmas estejam criadas.

Integridade de dados

Só para relembrar, a integridade de dados é o que garante a confiabilidade, a veracidade ou a consistência dos dados salvos em um banco.

Um exemplo da aplicação de uma regra de integridade já foi realizada anteriormente. Vamos relembrar:

```
ALTER TABLE Cidade
ADD CONSTRAINT FK_Cidade_UF FOREIGN KEY (IdUF) REFERENCES UF (IdUF)
```

Com esse comando adicionamos uma regra de integridade entre duas tabelas, onde o campo que é relacionado em uma tabela dependente, fica fortemente ligado a sua origem.

Abaixo uma tabale com os tipos de integridade existentes:

Tipo de Integridade	Tipo de Constraint
Domínio	Default
	Check
Entidade	Primary Key
	Unique
Referencial	Foreign Key
	Check

- Uma restrição pode ser criada no momento em que uma tabela:
 - É criada (CREATE TABLE) ou;
 - É alterada (ALTER TABLE).
- Além disso é possível adicionar constraints em tabelas com dados
- Podemos inserir constraints em uma ou várias colunas:
 - Em uma coluna: nível de coluna;
 - Em várias colunas: nível de tabela.

Tipos de Constraints

Neste tópico iremos estudar os tipos de constraints. Para exemplificar cada tipo, utilizaremos como exemplo as tabelas de nosso banco de dados.

1. Constraints NOT NULL

- Essa constraint assegura que não exista valores nulos em uma coluna;
- Essa informação é definida a nível de coluna (deve ser informado para cada coluna);
- Ela pode ser criada da seguinte maneira:

```
Create table [TipoTelefone]
(
    [IdTipoTelefone] Integer Identity(1,1) NOT NULL,
    [Descricao] Varchar(100) NOT NULL,
    Primary Key ([IdTipoTelefone])
)
```

ou então

```
Create table [TipoTelefone]
(
    [IdTipoTelefone] Integer Identity(1,1) NOT NULL,
    [Descricao] Varchar(100),
)

ALTER TABLE TipoTelefone
ALTER COLUMN Descricao Varchar(100) NOT NULL
```

2. Constraints PRIMARY KEY

- Essa constraint cria um índice exclusivo em uma coluna específica, onde os valores devem ser exclusivos e não podem ser nulos;
- É permitido apenas uma restrição PRIMARY KEY por tabela.

```
Create table [TipoTelefone]
(
    [IdTipoTelefone] Integer Identity(1,1) NOT NULL,
    [Descricao] Varchar(100) NOT NULL,
    Primary Key ([IdTipoTelefone])
)
```

ou então

```
Create table [TipoTelefone]
(
    [IdTipoTelefone] Integer Identity(1,1) Primary Key NOT NULL,
    [Descricao] Varchar(100) NOT NULL,
)
```

ou então

```
Create table [TipoTelefone]
(
    [IdTipoTelefone] Integer Identity(1,1) NOT NULL,
    [Descricao] Varchar(100) NOT NULL,
Constraint PK_TipoTelefone Primary Key ([IdTipoTelefone])
)
```

- É possível ainda realizar a criação da constraint após a criação de uma tabela, desde que respeite a regra de integridade da constraint.

```
ALTER TABLE TipoTelefone
ADD CONSTRAINT PK_TipoTelefone PRIMARY KEY (IdTipoTelefone)
```

- Também é possível realizar a criação de chaves compostas, somente em nível de tabela:

```
Create table [DisciplinaAluno]
(
    [IdDisciplina] Integer NOT NULL,
    [IdPessoa] Integer NOT NULL,
    [Nota] Integer NULL,
Primary Key ([IdDisciplina], [IdPessoa])
)
```

ou então

```
Create table [DisciplinaAluno]
(
    [IdDisciplina] Integer NOT NULL,
    [IdPessoa] Integer NOT NULL,
    [Nota] Integer NULL,
CONSTRAINT PK_NotaAlunoDisciplina Primary Key
([IdDisciplina], [IdPessoa])
)
```

3. Constraints DEFAULT

- As instruções default são utilizadas para definir um valor padrão para uma determinada coluna, permitindo também alguns valores permitidos pelo sistema.
- Essas constraints são aplicadas as instruções INSERT.

```
Create table [DisciplinaAluno]
(
    [IdDisciplina] Integer NOT NULL,
    [IdPessoa] Integer NOT NULL,
    [Nota] Integer Constraint [DF_NotaBase] Default 0 NOT NULL,
    CONSTRAINT PK_NotaAlunoDisciplina Primary Key
    ([IdDisciplina], [IdPessoa])
)
go
```

ou então

```
Create table [DisciplinaAluno]
(
    [IdDisciplina] Integer NOT NULL,
    [IdPessoa] Integer NOT NULL,
    [Nota] Integer Default 0 NOT NULL,
    CONSTRAINT PK_NotaAlunoDisciplina Primary Key
    ([IdDisciplina], [IdPessoa])
)
```

ou então

```
ALTER TABLE DisciplinaAlunoTeste
ADD CONSTRAINT DF_NotaBase DEFAULT 0 FOR Nota
```

4. Constraints CHECK

- Essas instruções podem ser utilizadas tanto para INSERT quanto para UPDATE.
- Além disso é possível utilizar outras colunas da mesma tabela para referenciar, porém não podem conter subconsultas.

```
Create table [Pessoa]
(
    [IdPessoa] Integer Identity(1,1) NOT NULL,
    [IdSexo] Integer NOT NULL,
```

```

[Nome] Varchar(200) NOT NULL,
[DataNascimento] Datetime CONSTRAINT CK_DataNascimento CHECK
(DataNascimento > '01-01-1990' AND DataNascimento < GETDATE())
NOT NULL,
[CPF] Varchar(11) NOT NULL,
[NomePai] Varchar(200) NULL,
[NomeMae] Varchar(200) NOT NULL,
Primary Key ([IdPessoa])
)

```

ou então

```

Create table [Pessoa]
(
    [IdPessoa] Integer Identity(1,1) NOT NULL,
    [IdSexo] Integer NOT NULL,
    [Nome] Varchar(200) NOT NULL,
    [DataNascimento] Datetime CHECK
(DataNascimento > '01-01-1990' AND DataNascimento < GETDATE())
NOT NULL,
    [CPF] Varchar(11) NOT NULL,
    [NomePai] Varchar(200) NULL,
    [NomeMae] Varchar(200) NOT NULL,
Primary Key ([IdPessoa])
)

```

ou ainda

```

ALTER TABLE Pessoa
ADD CONSTRAINT CK_DataNascimento CHECK (DataNascimento > '01-01-1990'
AND DataNascimento < GETDATE())
GO

```

5. Constraints UNIQUE

- As constraints UNIQUE impõem um índice exclusivo para o valor de um campo, permitindo valor nulo.
- É permitido várias restrições UNIQUE em uma tabela:

```

Create table [UF]
(
    [IdUF] Integer Identity(1,1) NOT NULL,
    [UF] Char(2) NOT NULL UNIQUE,
    [Estado] Varchar(100) NOT NULL,
Primary Key ([IdUF])
)

```

ou então

```

Create table [UF]
(
    [IdUF] Integer Identity(1,1) NOT NULL,

```

```

        [UF] Char(2) NOT NULL, UNIQUE (UF),
        [Estado] Varchar(100) NOT NULL,
Primary Key ([IdUF])
)

```

- É possível criar a constraint UNIQUE para mais de uma coluna:

```

Create table [UF]
(
    [IdUF] Integer Identity(1,1) NOT NULL,
    [UF] Char(2) NOT NULL,
    [Estado] Varchar(100) NOT NULL,
Primary Key ([IdUF]),
CONSTRAINT UK_Estado UNIQUE (UF, Estado)
)

```

ou então

```

ALTER TABLE UF
ADD CONSTRAINT UK_Estado UNIQUE (UF, Estado)

```

6. Constraints FOREIGN KEY

- Esse tipo de constraint deve fazer uma referência a uma restrição PRIMARY KEY ou UNIQUE.
- É utilizada para fornecer integridade referencial de uma ou várias colunas, e os índices não são criados automaticamente, como no caso de outras constraints.
- Os usuários devem ter permissões SELECT ou REFERENCES em tabelas que utilizam essa referência.

Essa é a referência a nível de coluna:

```

Create table [Cidade]
(
    [IdCidade] Integer Identity(1,1) NOT NULL,
    [IdUF] Integer FOREIGN KEY REFERENCES UF (IdUF) NOT NULL,
    [NomeCidade] Varchar(100) NOT NULL,
Primary Key ([IdCidade])
)

```

Essa é a referência a nível de tabela:

```
Create table [Cidade]
(
    [IdCidade] Integer Identity(1,1) NOT NULL,
    [IdUF] Integer NOT NULL,
    [NomeCidade] Varchar(100) NOT NULL
    FOREIGN KEY (IdUF) REFERENCES UF (IdUF)
Primary Key ([IdCidade])
)
```

ou então

```
ALTER TABLE Cidade
ADD CONSTRAINT FK_Cidade_UF FOREIGN KEY (IdUF) REFERENCES UF (IdUF)
```

Integridade Referencial em Cascata

- FOREIGN KEY vincula os dados da tabela filha enquanto exista referência de dados com a tabela mãe;
- REFERENCES é utilizado para indicar a tabela e qual coluna é utilizada como referência na tabela mãe;
- ON DELETE CASCADE: Ao efetuar uma exclusão de um valor na tabela mãe, todas as linhas que dependiam desse valor também são excluídas;
- ON UPDATE CASCADE: Ao efetuar uma atualização de um valor na tabela mãe, todas as linhas que dependiam desse valor também são atualizadas.

```
Create table [Cidade]
(
    [IdCidade] Integer Identity(1,1) NOT NULL,
    [IdUF] Integer FOREIGN KEY REFERENCES UF (IdUF) ON DELETE
CASCADE NOT NULL,
    [NomeCidade] Varchar(100) NOT NULL,
Primary Key ([IdCidade])
)
```

ou então

```
ALTER TABLE Cidade
ADD CONSTRAINT FK_Cidade_Estado
FOREIGN KEY([IdUF])
```

```
REFERENCES [dbo].[UF] ([IdUF])
ON DELETE CASCADE
ON UPDATE CASCADE
GO
```

Removendo uma constraint

Para remover uma constraint utilizamos a seguinte instrução:

```
ALTER TABLE UF
DROP CONSTRAINT PK_UF
```

Para exclusão de constraints PRIMARY KEY E UNIQUE KEY, suas referências devem ser removidas, não podendo haver nenhuma outra constraint de FOREIGN KEY. Para efetuar a exclusão:

- Remova primeiro a constraint de FOREIGN KEY.
 - No nosso caso, como a tabela “Cidade” referencia a tabela “UF”, devmos excluir a constraint FOREIGN KEY “FK_Cidade_UF” na tabela “Cidade”:

```
ALTER TABLE Cidade
DROP CONSTRAINT FK_Cidade_UF
```

- E depois excluir a constraint “PK_UF” na tabela “UF”:

```
ALTER TABLE UF
DROP CONSTRAINT PK_UF
```

Desativando uma constraint

- Utilizado para processamento de operações em lote (grande volume de dados);
- Utilizando essa instrução é ignorada a verificação da relação entre as tabelas;
- Só é possível desativar as constraints FOREIGN KEY e CHECK.

```
ALTER TABLE Cidade
WITH NOCHECK
ADD CONSTRAINT FK_Cidade_UF
FOREIGN KEY (IdUF)
REFERENCES UF (IdUF)
```

Criando uma constraint FOREIGN KEY dessa maneira estamos querendo dizer que não será verificado se existe integridade de relacionamento entre a tabela “Cidade” e a tabela “UF”.

Para desativar uma constraint de chave estrangeira:

```
ALTER TABLE Cidade
NOCHECK CONSTRAINT FK_Cidade_UF
```

Após essa modificação é possível realizar a inserção de um dado que não obedeça a regra de integridade:

```
INSERT INTO Cidade (IdUF, NomeCidade)
VALUES (7, 'Maria Mole')
```

Para reativar a constraint, executamos a instrução:

```
ALTER TABLE Cidade
CHECK CONSTRAINT FK_Cidade_UF
```

Agora, se tentarmos efetuar uma inserção que não obedeça a regra de integridade:

```
INSERT INTO Cidade (IdUF, NomeCidade)
VALUES (8, 'Pão de Queijo')
```

Será exibido o erro:

```
Msg 547, Level 16, State 0, Line 1
The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Cidade_UF". The
conflict occurred in database "Academico", table "dbo.UF", column 'IdUF'.
The statement has been terminated.
```

Informando que não é possível realizar inserção, pois o IdUF que estamos informando não existe na tabela “UF”.

Criar Views

- São instruções SELECT que são armazenadas no banco;
- O acesso delas é via select.

A instrução para criação de uma View é:

```
DROP VIEW VW_Pessoa
GO
```

```
CREATE VIEW VW_Pessoa AS
SELECT IdPessoa, Nome, NomeMae, NomePai, CPF
FROM Pessoa
```

- As duas primeiras linhas são para a exclusão da view, caso ela exista.
- Após isso, criamos uma view com o nome de “VW_Pessoa” que busca na tabela “Pessoa” os dados “IdPessoa”, “Nome”, “NomeMae”, “NomePai”, “CPF”.
- Além disso é possível adicionar dados em uma view, e assim, adicionar valores em uma tabela:

```
CREATE VIEW VW_UF AS
SELECT IdUF, UF, Estado FROM UF

INSERT INTO VW_UF (UF, Estado)
VALUES ('SC', 'Santa Catarina')

SELECT * FROM VW_UF
```

- Inicialmente criamos uma nova view chamada “VW_UF”;
- Após isso realizamos um insert na própria view, lembrando que é necessário obedecer as constraints existentes na tabela para fazer uma nova inserção;
- Após isso realizamos uma consulta, para ver que o novo valor consta em nossa tabela “UF”.

Também podemos utilizar a view para gerar consultas através de uniões entre as tabelas:

```
CREATE VIEW VW_Cidade_Estado AS
SELECT C.NomeCidade, U.Estado, U.UF
FROM UF AS U INNER JOIN Cidade AS C ON U.IdUF = C.IdUF

SELECT * FROM VW_Cidade_Estado
```

- Assim estamos criando uma view chamanda “VW_Cidade_Estado”, que realiza a união entre a tabela “Estado” e a tabela “Cidade”, que são ligadas através do campo “IdUF”.
- No final teremos uma view que nos apresenta o nome da cidade, o estado e a unidade federativa de cada estado.

Atualização do banco de dados

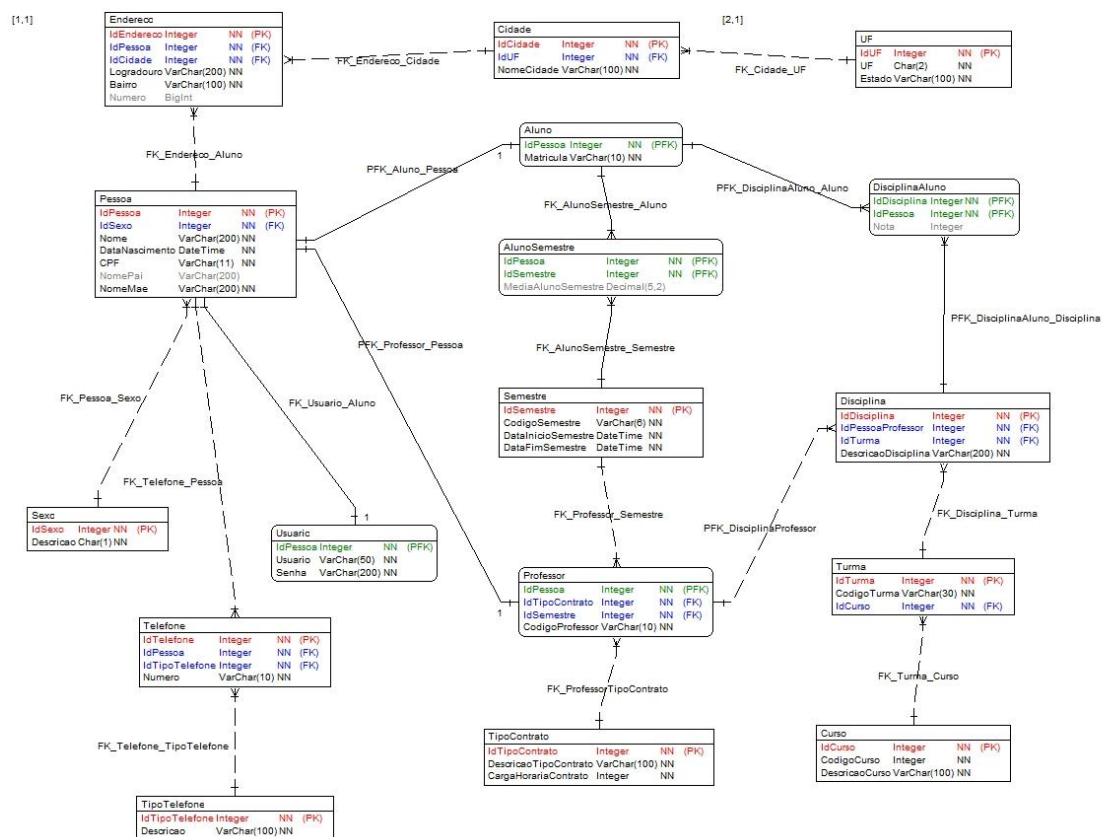
De acordo com os novos requisitos indentificados e trabalhados em sala de aula, irei disponibilizar abaixo o DER e o Script para geração do banco atualizado:

Requisitos

- Cada professor deve possuir obrigatoriamente um contrato de trabalho;
- Tanto o aluno quanto o professor devem possuir um usuário;
- O usuário de aluno será sua matrícula;
- O professor deve ter como usuário o código do professor;
- O sistema deve registrar uma senha padrão para acesso ao sistema, já criptografada em MD5;
- Uma turma pode ou não ter uma ou mais disciplinas e a disciplina pode ser de uma ou mais turmas
- Devem ser registrados os Cursos da instituição.
- As turmas são vinculadas a um curso. As turmas são apenas de um curso e um curso deve possuir no mínimo uma turma;

- É necessário realizar o cadastro de um semestre. O semestre deve possuir um código de identificação, a data início e a data fim do semestre.
- Tanto o professor quanto o aluno devem estar matriculados em um semestre.
- Uma disciplina não precisa estar vinculada nem a um professor e nem a uma turma.

DER



Script do banco

```
CREATE TABLE [Pessoa]
(
    [IdPessoa] INTEGER IDENTITY(1,1) NOT NULL,
    [IdSexo] INTEGER NOT NULL,
    [Nome] VARCHAR(200) NOT NULL,
    [DataNascimento] DATETIME NOT NULL,
    [CPF] VARCHAR(11) NOT NULL,
    [NomePai] VARCHAR(200) NULL,
```

```

        [NomeMae] VARCHAR(200) NOT NULL,
PRIMARY KEY ([IdPessoa])
)
GO

CREATE TABLE [Disciplina]
(
    [IdDisciplina] INTEGER IDENTITY(1,1) NOT NULL,
    [IdPessoaProfessor] INTEGER NOT NULL,
    [IdTurma] INTEGER NOT NULL,
    [DescricaoDisciplina] VARCHAR(200) NOT NULL,
PRIMARY KEY ([IdDisciplina])
)
GO

CREATE TABLE [DisciplinaAluno]
(
    [IdDisciplina] INTEGER NOT NULL,
    [IdPessoa] INTEGER NOT NULL,
    [Nota] INTEGER NULL,
PRIMARY KEY ([IdDisciplina], [IdPessoa])
)
GO

CREATE TABLE [Endereco]
(
    [IdEndereco] INTEGER IDENTITY(1,1) NOT NULL,
    [IdPessoa] INTEGER NOT NULL,
    [IdCidade] INTEGER NOT NULL,
    [Logradouro] VARCHAR(200) NOT NULL,
    [Bairro] VARCHAR(100) NOT NULL,
    [Numero] BIGINT NULL,
PRIMARY KEY ([IdEndereco])
)
GO

CREATE TABLE [Cidade]
(
    [IdCidade] INTEGER IDENTITY(1,1) NOT NULL,
    [IdUF] INTEGER NOT NULL,
    [NomeCidade] VARCHAR(100) NOT NULL,
PRIMARY KEY ([IdCidade])
)
GO

CREATE TABLE [UF]
(
    [IdUF] INTEGER IDENTITY(1,1) NOT NULL,
    [UF] CHAR(2) NOT NULL,
    [Estado] VARCHAR(100) NOT NULL,
PRIMARY KEY ([IdUF])
)
GO

```

```
CREATE TABLE [Usuario]
(
    [IdPessoa] INTEGER NOT NULL,
    [Usuario] VARCHAR(50) NOT NULL,
    [Senha] VARCHAR(200) NOT NULL,
    PRIMARY KEY ([IdPessoa])
)
GO

CREATE TABLE [Aluno]
(
    [IdPessoa] INTEGER NOT NULL,
    [Matricula] VARCHAR(10) NOT NULL,
    PRIMARY KEY ([IdPessoa])
)
GO

CREATE TABLE [Telefone]
(
    [IdTelefone] INTEGER IDENTITY(1,1) NOT NULL,
    [IdPessoa] INTEGER NOT NULL,
    [IdTipoTelefone] INTEGER NOT NULL,
    [Numero] VARCHAR(10) NOT NULL,
    PRIMARY KEY ([IdTelefone])
)
GO

CREATE TABLE [TipoTelefone]
(
    [IdTipoTelefone] INTEGER IDENTITY(1,1) NOT NULL,
    [Descricao] VARCHAR(100) NOT NULL,
    PRIMARY KEY ([IdTipoTelefone])
)
GO

CREATE TABLE [Sexo]
(
    [IdSexo] INTEGER IDENTITY(1,1) NOT NULL,
    [Descricao] CHAR(1) NOT NULL,
    PRIMARY KEY ([IdSexo])
)
GO

CREATE TABLE [Professor]
(
    [IdPessoa] INTEGER NOT NULL,
    [IdTipoContrato] INTEGER NOT NULL,
    [IdSemestre] INTEGER NOT NULL,
    [CodiGOProfessor] VARCHAR(10) NOT NULL,
    PRIMARY KEY ([IdPessoa])
)
GO
```

```

CREATE TABLE [TipoContrato]
(
    [IdTipoContrato] INTEGER IDENTITY(1,1) NOT NULL,
    [DescricaoTipoContrato] VARCHAR(100) NOT NULL,
    [CargaHorariaContrato] INTEGER NOT NULL,
    PRIMARY KEY ([IdTipoContrato])
)
GO

CREATE TABLE [Turma]
(
    [IdTurma] INTEGER IDENTITY(1,1) NOT NULL,
    [CodiGOTurma] VARCHAR(30) NOT NULL,
    [IdCurso] INTEGER NOT NULL,
    PRIMARY KEY ([IdTurma])
)
GO

CREATE TABLE [Curso]
(
    [IdCurso] INTEGER IDENTITY(1,1) NOT NULL,
    [CodiGOCurso] INTEGER NOT NULL,
    [DescricaoCurso] VARCHAR(100) NOT NULL,
    PRIMARY KEY ([IdCurso])
)
GO

CREATE TABLE [Semestre]
(
    [IdSemestre] INTEGER IDENTITY (1,1) NOT NULL,
    [CodiGOSemestre] VARCHAR(6) NOT NULL,
    [DataInicioSemestre] DATETIME NOT NULL,
    [DataFimSemestre] DATETIME NOT NULL,
    PRIMARY KEY ([IdSemestre])
)
GO

CREATE TABLE [AlunoSemestre]
(
    [IdPessoa] INTEGER NOT NULL,
    [IdSemestre] INTEGER NOT NULL,
    [MediaAlunoSemestre] DECIMAL(5,2) NULL,
    PRIMARY KEY ([IdPessoa],[IdSemestre])
)
GO

ALTER TABLE [Endereco]
ADD FOREIGN KEY([IdPessoa]) REFERENCES [Pessoa] ([IdPessoa])
GO
ALTER TABLE [Usuario]
ADD FOREIGN KEY([IdPessoa]) REFERENCES [Pessoa] ([IdPessoa])

```

```

GO
ALTER TABLE [Aluno]
ADD FOREIGN KEY([IdPessoa]) REFERENCES [Pessoa] ([IdPessoa])
GO
ALTER TABLE [Telefone]
ADD FOREIGN KEY([IdPessoa]) REFERENCES [Pessoa] ([IdPessoa])
GO
ALTER TABLE [Professor]
ADD FOREIGN KEY([IdPessoa]) REFERENCES [Pessoa] ([IdPessoa])
GO
ALTER TABLE [DisciplinaAluno]
ADD FOREIGN KEY([IdDisciplina]) REFERENCES [Disciplina]
([IdDisciplina])
GO
ALTER TABLE [Endereco]
ADD FOREIGN KEY([IdCidade]) REFERENCES [Cidade] ([IdCidade])
GO
ALTER TABLE [Cidade]
ADD FOREIGN KEY([IdUF]) REFERENCES [UF] ([IdUF])
GO
ALTER TABLE [DisciplinaAluno]
ADD FOREIGN KEY([IdPessoa]) REFERENCES [Aluno] ([IdPessoa])
GO
ALTER TABLE [AlunoSemestre]
ADD FOREIGN KEY([IdPessoa]) REFERENCES [Aluno] ([IdPessoa])
GO
ALTER TABLE [Telefone]
ADD FOREIGN KEY([IdTipoTelefone]) REFERENCES [TipoTelefone]
([IdTipoTelefone])
GO
ALTER TABLE [Pessoa]
ADD FOREIGN KEY([IdSexo]) REFERENCES [Sexo] ([IdSexo])
GO
ALTER TABLE [Disciplina]
ADD FOREIGN KEY([IdPessoaProfessor]) REFERENCES [Professor]
([IdPessoa])
GO
ALTER TABLE [Professor]
ADD FOREIGN KEY([IdTipoContrato]) REFERENCES [TipoContrato]
([IdTipoContrato])
GO
ALTER TABLE [Disciplina]
ADD FOREIGN KEY([IdTurma]) REFERENCES [Turma] ([IdTurma])
GO
ALTER TABLE [Turma]
ADD FOREIGN KEY([IdCurso]) REFERENCES [Curso] ([IdCurso])
GO
ALTER TABLE [Professor]
ADD FOREIGN KEY([IdSemestre]) REFERENCES [Semestre] ([IdSemestre])
GO
ALTER TABLE [AlunoSemestre]
ADD FOREIGN KEY([IdSemestre]) REFERENCES [Semestre] ([IdSemestre])
GO

```

Triggers

Trigger é um bloco de comandos Transact-SQL que é executado quando ocorre um comando INSERT, DELETE ou UPDATE for executando em um banco de dados.

Alguns exemplos de aplicações:

- Validações;
- Rotinas;
- Consistência de dados;
- Análise de dados;
- Alteração em tabelas que referenciam uma tabela que está sendo trabalhada.

Existem três aspectos importantes de um gatilho:

- Evento: uma alteração que ativa a trigger;
- Condição: consulta ou teste executado quando a trigger é ativada;
- Ação: procedimento executado quando a condição é atendida.

Abaixo, uma Trigger criada para nosso banco de dados “Acadêmico”. Essa Trigger será executada sempre que um novo aluno for inserido ou atualizado no banco:

```
USE [Academico]
GO

/*CRIAÇÃO DA TRIGGER NA TABELA "Aluno",
QUE SERÁ EXECUTADA SEMPRE QUE UM VALOR
FOR ADICIONADO OU ALTERADO NA TABELA*/
CREATE TRIGGER CriarUsuario
ON Aluno
/*DEFINIÇÃO DE QUANDO ELA SERÁ EXECUTADA,
NESSE CASO APÓS UM INSERT OU UPDATE*/
AFTER INSERT, UPDATE AS

/*DECLARAÇÃO DAS VARIÁVEIS QUE SERÃO NECESSÁRIAS*/
DECLARE @IdPessoa INT
DECLARE @Matricula VARCHAR(10)
DECLARE @verificaUsuario BIT
DECLARE @buscaUsuario VARCHAR(50)
DECLARE @Mensagem VARCHAR(8000)

/*BUSCA DE DADOS NA TABELA TEMPORÁRIA "INSERTED",
```

```

QUE JÁ É CRIADA PELO PRÓPRIO SQL SERVER*/
SELECT @IdPessoa = IdPessoa, @Matricula = Matricula
FROM INSERTED

SELECT @buscaUsuario = Usuario

FROM Usuario
WHERE Usuario = @Matricula

/*CASO JÁ EXISTA UM USUÁRIO NO BANCO PARA A MATRÍCULA
INFORMADA, A TRIGGER RETORNARÁ UM ERRO*/
IF (@buscaUsuario IS NOT NULL)
BEGIN
    SELECT @Mensagem = 'Já existe um usuário para a matrícula
informada';
    RAISERROR (@Mensagem, 16, 10)
    RETURN
END

/*CASO CONTRÁRIO, UM USUÁRIO SERÁ CRIADO PARA UM ALUNO ASSIM
QUE ELE SEJA INSERIDO OU ATUALIZADO NO BANCO DE DADOS*/
ELSE --(@buscaUsuario IS NULL)
BEGIN
    INSERT INTO Usuario (IdPessoa, Usuario, Senha)
    VALUES (@IdPessoa, @Matricula, HASHBYTES('MD5', '123456'))
END
RETURN

GO

```

Vamos agora as definições das palavras reservadas de uma Trigger:

- `CREATE TRIGGER` CriarUsuario é o comando para se criar uma nova Trigger.
 - Caso uma Trigger já esteja criada é possível alterar ela através do comando `ALTER TRIGGER` Nome_Trigger criada;
 - O comando para exclusão de uma Trigger é `DROP TRIGGER` Nome_Trigger ;
- `ON` Aluno `AFTER INSERT, UPDATE` define em qual tabela a Trigger será criada e em quais condições ela será executada. Exemplo:
 - Em nossa Trigger, caso efetuarmos um `INSERT INTO ALUNO` ou `UPDATE ALUNO` essa Trigger será executada.
- `DECLARE` @IdPessoa `INT` e as demais declarações de variáveis são utilizadas para criar variáveis locais que serão utilizadas como apoio na estrutura da Trigger.
 - DICA: Sempre trabalhe com variáveis que são compatíveis com o valor que ela irá receber, tanto em tipo quanto em limite de

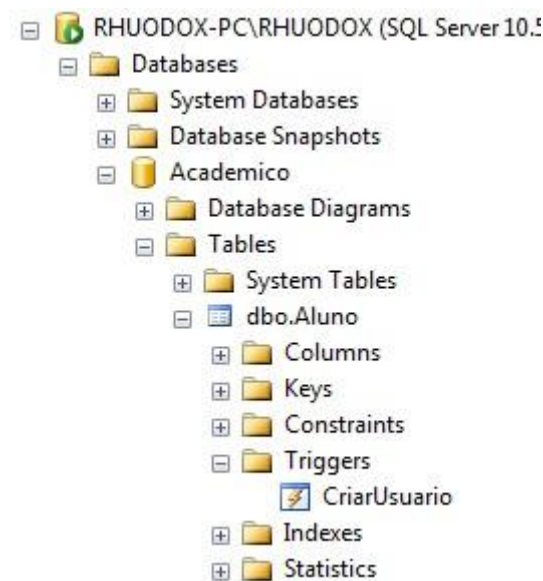
tamanho. Além disso, crie variáveis com nome amigáveis, assim fica mais fácil de se situar durante a criação ou mapeamento de uma Trigger ou qualquer estrutura que necessite da criação de variáveis.

- `SELECT @IdPessoa = IdPessoa, @Matricula = Matricula FROM INSERTED` já é nossa conhecida. A diferença existente é a origem dos dados. A tabela `INSERTED` é uma tabela temporária criada no banco de dados. Ela é utilizada para buscar os dados que são manipulados durante a execução de uma Trigger.
- O SQL Server mantém duas tabelas para controle das transações do banco: `INSERTED` E `DELETED`.
 - Ao executar o comando `INSERT`: O novo registro será armazenado na tabela `INSERTED`;
 - Ao executar o comando `DELETE`: O registro excluído será armazenado na tabela `DELETED`;
 - Ao executar o comando `UPDATE`: O novo registro será armazenado na tabela `INSERTED` e o antigo é armazenado na tabela `DELETED`.
- O comando `IF (@buscaUsuario IS NOT NULL) BEGIN SELECT @Mensagem = 'Já existe um usuário para a matrícula informada'; RAISERROR (@Mensagem, 16, 10) RETURN END` retorna uma mensagem para o utilizador do banco, informando que uma condição não foi verdadeira. No nosso exemplo, se a pessoa já possuir um usuário cadastrado, ela não pode inserir mais um.
- Caso a condição seja atendida, então temos o `ELSE --(@buscaUsuario IS NULL) BEGIN INSERT INTO Usuario (IdPessoa, Usuario, Senha) VALUES (@IdPessoa, @Matricula, HASHBYTES('MD5', '123456')) END` que realiza a inserção de um novo usuário na tabela “Usuario”.

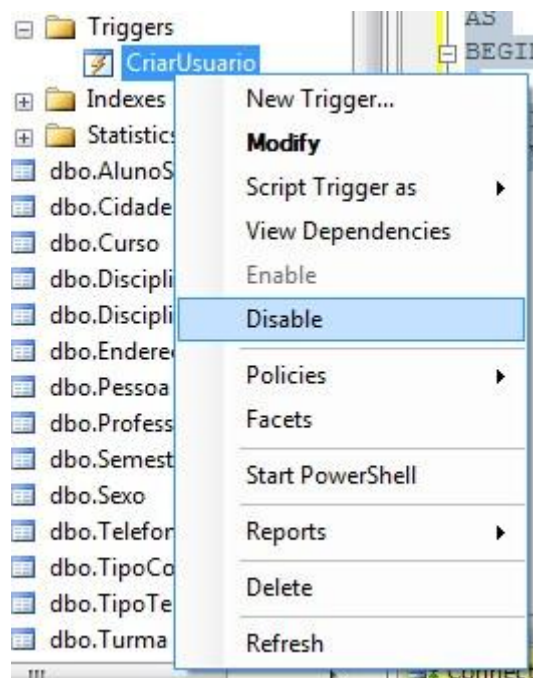
DICA: Um post interessante como auxílio para estudos sobre Triggers:

<http://www.devmedia.com.br/introducao-a-triggers/1695>

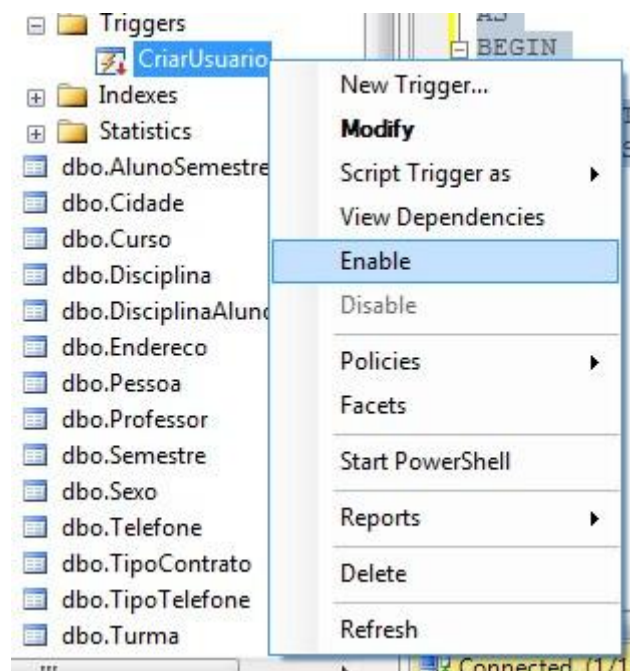
As Triggers criadas podem ser visualizadas pela própria tabela para qual ela é criada:



Além disso é possível ativar ou desativar uma Trigger. Para isso basta clicar com o botão direito sobre a Trigger e clicar em “Disable” (ou “Desabilitar” dependendo do idioma):



Assim ela ficará com uma seta vermelha em seu ícone, indicando que ela está desabilitada. Para habilitá-la basta clicar com o botão direito sobre ela e escolher a opção “Enable” (ou “Habilitar”, dependendo do idioma):



Legal ☺! Vamos para o próximo tópico: Stored Procedure! \o/

Stored Procedures

Stored Procedure é um trecho de código Transact SQL armazenado no banco de dados, que pode receber parâmetros e retornar valores. Quando uma Stored Procedure é executada, é realizada uma compilação da mesma.

Resumindo, Stored Procedure basicamente pode ser uma ou mais ações que são salvas no banco de dados e que podem ser solicitadas sempre que necessário.

As SP's podem ser utilizadas para muitos fins, como:

- Executar comandos INSERT;
- Executar comandos DELETE;
- Executar comandos UPDATE;
- Executar comandos SELECT;
- Efetuar controle em lista de registros (Cursor);
- Efetuar a execução de outras SP's.

Utilizando o banco “Acadêmico”, iremos criar algumas SP’S.

```
CREATE PROCEDURE SP_InseroSexo
(
    @Descricao CHAR (1)
)
AS
BEGIN

    INSERT INTO Sexo (Descricao)
    VALUES (@Descricao)

END

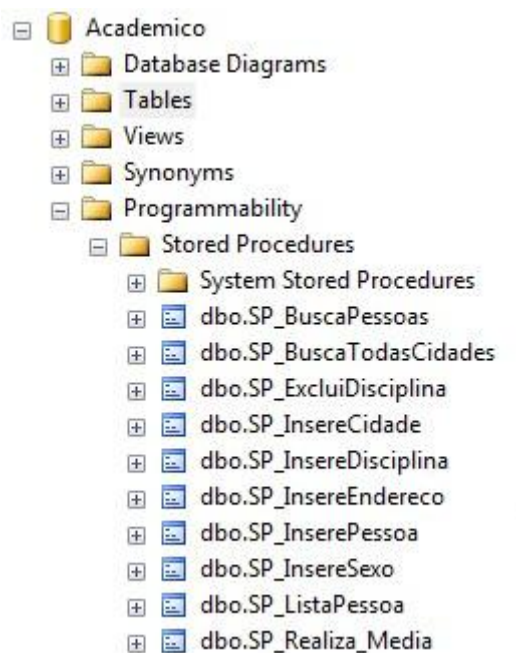
GO
```

A Stored Procedure acima é acionada para realizar um INSERT na tabela “Sexo”.

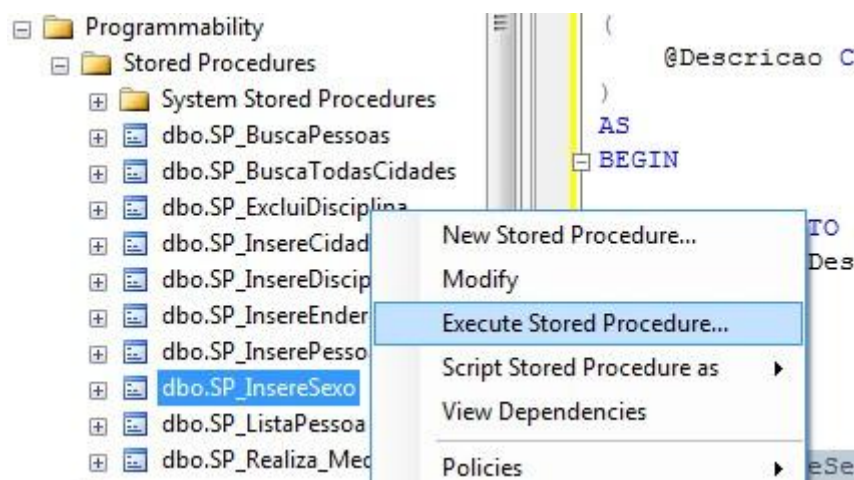
- O comando `CREATE PROCEDURE SP_InseroSexo` é utilizado para definir que estamos criando uma Stored Procedure e que o nome dela será “SP_InseroSexo”;
- Dentro dos parênteses informamos qual é(são) a(s) variável (eis) que serão utilizadas em nossa Stored Procedure. No caso dessa SP teremos apenas um campo, que no caso será referenciado pela variável `@Descricao CHAR (1)`;
- No bloco `AS BEGIN ... END` informamos qual ou quais ações iremos realizar. No caso dessa SP iremos realizar um insert, então iremos definir que para o campo “Descricao” na tabela “Sexo”, o valor a ser inserido será o da variável “@Descrição”.

A execução da Stored Procedure pode ser feita de duas maneiras:

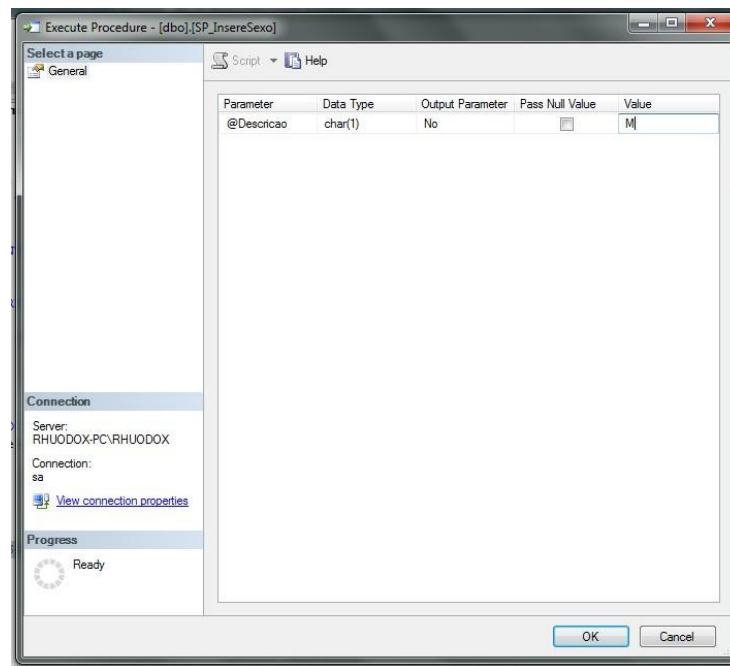
- `EXEC SP_InseroSexo 'M'` é o comando realizado a nível de script de banco, onde é passado o valor da variável a ser inserida;
- Outra maneira de executar a SP é diretamente pela tabela. Primeiramente abrimos o local onde se encontram as SP’s:



- Após isso clicamos com o botão direito sobre a SP que queremos executar e escolhemos a opção “Execute Stored Procedure...”:



- Depois disso informamos o valor para a(s) variável (eis) da SP e clicamos em OK. Assim a SP será executada:



Exemplo de Stored Procedure para UPDATE

```
CREATE PROCEDURE SP_AlteraSexo
(
    @Descricao CHAR (1)
)
AS
BEGIN

    UPDATE INTO Sexo (Descricao)
    VALUES (@Descricao)

END

GO
```

Exemplo de Stored Procedure para DELETE

```
CREATE PROCEDURE SP_ExcluiDisciplina
(
    @NomeDisciplina VARCHAR (100)
)
AS
BEGIN
    DELETE FROM Disciplina WHERE
    DescricaoDisciplina = @NomeDisciplina
END

GO
```

Exemplo de Stored Procedure para SELECT

```
CREATE PROCEDURE [dbo].[SP_BuscaTodasCidades]
(
    @IdUF INTEGER
)
AS
BEGIN
    SELECT * FROM Cidade WHERE IdUF = @IdUF
END

GO
```

Outro exemplo de Stored Procedure para INSERT (mais legal! ☺)

```
CREATE PROCEDURE [dbo].[SP_Realiza_Media]
(
    @CodigoSemestre VARCHAR (6),
    @Matricula VARCHAR (10)
)
AS
BEGIN
    BEGIN TRANSACTION TR_Valida_Dados

    DECLARE @IdPessoa INT
    DECLARE @MediaAluno DECIMAL (5,2)
    DECLARE @Mensagem VARCHAR (200)

    SELECT @IdPessoa = Idpessoa FROM Aluno WHERE Matricula =
@Matricula

    SELECT @MediaAluno = AVG(nota) FROM DisciplinaAluno WHERE
IdPessoa = @IdPessoa

    IF (@IdPessoa IS NULL OR @MediaAluno IS NULL)
    BEGIN
        SELECT @Mensagem = 'Não é permitido valor nulo para essa
operação';

        RAISERROR (@Mensagem, 16, 10)
        ROLLBACK TRANSACTION TR_Valida_Dados

        RETURN
    END

    ELSE
    BEGIN
        UPDATE AlunoSemestre SET MediaSemestre = @MediaAluno WHERE
IdPessoa = @IdPessoa

        COMMIT TRANSACTION TR_Valida_Dados
    END

END

GO
```

Na SP acima estamos realizando a média da nota do semestre do aluno na tabela “Aluno”, atualizando o campo “MediaSemestre” com o valor calculado.

Uma coisa nova que não existe nas SP's anteriores é o uso de um controle de transação.

Transações em SP's são importantes para garantir a integridade e segurança da execução de uma alteração no banco, onde ela garante que somente será alterado o banco se não houver erros durante o controle de uma transação, ou seja, só vai salvar se tudo der certo.

- `BEGIN TRANSACTION TR_Valida_Dados` é utilizado para iniciar a transação. “TR_Valida_Dados” é o nome para transação. Esse nome ajuda a identificar onde uma transação começa e termina.
- `ROLLBACK TRANSACTION TR_Valida_Dados` é utilizado para reverter a ação efetuada caso aconteça algum erro. Assim, tudo que esteja dentro de uma transação é desconsiderado.
- `COMMIT TRANSACTION TR_Valida_Dados` é utilizado para submeter as alterações realizadas em uma transação. Ao fazer isso, todas as comandos realizados são registrados no banco.

Cursor

Um Cursor é um comando que permite realizar operações em linhas individuais de um resultado, ou seja, ao executarmos, por exemplo, um select que retorne mais de um item, podemos utilizar um Cursor para realizarmos ações sobre cada linha de resultado da consulta, podendo ser manipulada de qualquer maneira. Abaixo algumas definições:

- **INSENSITIVE:** na abertura do cursor, o resultado é armazenado em uma tabela temporária, ou seja, as modificações posteriores a sua abertura não serão conhecidas;

- **SCROLL:** todas as operações de movimentação poderão ser realizadas, não somente as definidas pelo padrão ANSI/92 (movimentação à frente);
- **READ ONLY:** não permite atualizações utilizando o cursor;
- **UPDATE [OF colunas]:** permite atualizações em todas (comportamento padrão) ou somente algumas colunas;
- Monta e disponibiliza o resultado do cursor, sintaxe:
 - **OPEN** nome do cursor
- **@@CURSOR_ROWS** indica o número de linhas que atenderam a sua consulta.
- O Comando **FETCH** realiza a movimentação em um cursor, permitindo percorrê-lo linha a linha, sintaxe:
 - **FETCH** [[**NEXT** | **PRIOR** | **FIRST** | **LAST** | **ABSOLUTE** n | **RELATIVE** n] **FROM**] nome_do_cursor [**INTO** @variável1, @variável2, ...].
- A Variável **@@FETCH_STATUS** é utilizada para indicar o estado da movimentação:
 - **@@FETCH_STATUS** = 0: movimentação realizado com sucesso;
 - **@@FETCH_STATUS** = -1: cursor está na linha inicial ou final;
 - **@@FETCH_STATUS** = -2: a linha recuperada não é válida (foi modificada) em um cursor não **INSENSITIVE**;
- **NEXT:** move para a próxima linha do cursor ou para a primeira, se o cursor foi recém aberto;
- **PRIOR:** move para a linha anterior;
- **FIRST** e **LAST:** move para a primeira e última linhas, respectivamente;
- **ABSOLUTE** n: move para a linha de posição n no cursor (se for positivo, a contagem inicia na primeira linha, se negativo, na última);
- **RELATIVE** n: move para n linhas para frente (positivo) ou para trás (negativo) a partir da posição atual;
- **PRIOR, FIRST, LAST, ABSOLUTE** n e **RELATIVE** n só podem ser realizados com cursores **SCROLL**;

- INTO @variavel1[, @variavel2...]: permite associar cada coluna do cursor a uma variável declarada;
- Cada variável listada no comando FETCH deverá estar relacionada a uma coluna do cursor;
- A variável deve possuir o mesmo tipo da coluna, não sendo realizadas conversões implícitas.
- Um cursor pode ser fechado através do comando CLOSE, sintaxe:
 - CLOSE nome_do_cursor;
 - Um cursor fechado mantém as estruturas de dados criadas através do comando DECLARE CURSOR, ou seja, pode ser reaberto através do comando OPEN;
- Um cursor pode ser desalocado, ou seja, ter eliminadas as estruturas de dados criadas através DEALLOCATE nome_do_cursor;
- Um cursor desalocado não pode mais ser reaberto.

Vamos criar um Cursor para exercitarmos um pouco. O Cursor abaixo lista alguns dados da tabela “Pessoa”:

```
CREATE PROCEDURE SP_ListaPessoa AS
BEGIN
DECLARE @Nome VARCHAR (200)
DECLARE @DataNascimento DATETIME
DECLARE @Mensagem VARCHAR (500)

DECLARE CursorLista SCROLL CURSOR
FOR
    SELECT nome, datanascimento
    FROM pessoa
    FOR READ ONLY

    OPEN CursorLista

    FETCH NEXT FROM CursorLista INTO @Nome, @DataNascimento
    WHILE @@FETCH_STATUS = 0
    BEGIN
        SELECT @Mensagem = 'O nome da pessoa é: ' +
LTRIM(@Nome) +
        ' - A data de nascimento é: ' + CONVERT(CHAR(10),
@DataNascimento, 103)
        PRINT @Mensagem
        FETCH NEXT FROM CursorLista INTO @Nome,
@DataNascimento
    END
DEALLOCATE CursorLista
END
GO
```

- `CREATE PROCEDURE SP_ListaPessoa` como já sabemos, realiza a criação da SP que irá utilizar um cursor;
- Após isso declaramos as variáveis que serão utilizadas. Essas variáveis tem dependência com os dados que buscamos dentro do cursos, ou seja, elas que irão receber os valores;
- `DECLARE CursorLista SCROLL CURSOR` é o comando para criarmos um novo cursor;
- ```
FOR SELECT Nome, DataNascimento
FROM Pessoa
FOR READ ONLY
OPEN CursorLista
```

Inicia a busca dos dados que iremos trabalhar. Nesse caso, estamos buscando os campos “Nome” e “DataNascimento” da tabela “Pessoa”. Após isso Abrimos o Cursor com o nome de “CursorLista”, ou seja, iremos começar a percorrer os dados da consulta através de um Cursor.

- ```
FETCH NEXT FROM CursorLista INTO @Nome, @DataNascimento
WHILE @@FETCH_STATUS = 0
BEGIN
    SELECT @Mensagem = 'O nome da pessoa é: ' +
        LTRIM(@Nome) +
        ' - A data de nascimento é: ' + CONVERT(CHAR(10),
        @DataNascimento, 103)
    PRINT @Mensagem
```

Para uma linha buscada da consuta, iremos inserir na variável “@Nome” o valor do campo “Nome” e na variável “@DataNascimento” o valor do campo “DataNascimento”. O comando WHILE diz que enquanto a movimentação for realizada com sucesso, iremos retornar a mensagem abaixo, apresentando os dados buscados na tabela “Pessoa” e apresentaremos essa mensagem com o comando PRINT.

- ```
FETCH NEXT FROM CursorLista INTO @Nome, @DataNascimento
END
```

Após isso pegaremos a próxima linha do Cursor, e o mesmo será feito até que todas as linhas sejam percorridas.

- ```
DEALLOCATE CursorLista
END
```

O comando DEALLOCATE elimina ou fecha o Cursor que estava sendo trabalhado.

O próximo e último tópico a ser trabalhado sobre programação em banco de dados será Funções.

Funções

As Funções são instruções criadas para auxiliar na busca e no tratamento de informação dentro de um banco de dados, onde é possível receber mais de um parâmetro.

Elas podem ser utilizadas para criar condições e para formatar e apresentar dados por exemplo, nunca modificando o estado do banco de dados.

Basicamente, podem ser de três tipos:

1. Scalar Function: Função que retorna um valor simples, como em muitas linguagens de programação existentes;
 - Utilizada para apresentar resultados pequenos.
2. In-Line Table-Valued Function: Função que retorna obrigatoriamente os valores em forma de tabelas;
 - Pode ser usada ao invés de Stored Procedure para retornar tabelas.
3. Multi-Statement Function: Função que retorna os valores em forma de tabela, onde é possível atualizar a tabela em que os dados são armazenados.
 - Pode ser usada para criar views parametrizadas.

Vamos demonstrar um exemplo de cada função:

Scalar Function

```
CREATE FUNCTION FormatarTelefone (@numeroTelefone VARCHAR (10))
RETURNS VARCHAR(13) AS
BEGIN
    DECLARE @telefone VARCHAR(13)
    SET @telefone = '(' + SUBSTRING(@numeroTelefone,1,2) + ')'
+ SUBSTRING(@numeroTelefone,3,4)
    + '-' + SUBSTRING(@numeroTelefone,7,4)

    RETURN @telefone
END
```

Essa função cria uma máscara para os telefones cadastrados. Como temos telefones no formato “3438230300”, ao criarmos uma função para organizar o campo “Numero” da tabela “Telefone”, teremos o seguinte resultado: “(34) 3823-0300”.

Para executar a Função basta realizar o seguinte comando:

```
SELECT dbo.FormatarTelefone(Numero) FROM Telefone
```

Como já vimos acima, uma função não altera os dados de um banco, e só pode ser utilizado para tratar os dados de uma consulta.

In-Line Table-Valued Function

```
CREATE FUNCTION RetornaPessoa(@Nome VARCHAR(100))
RETURNS TABLE
AS

RETURN (SELECT Nome , dbo.formatartelefone (t.Numero)
AS Telefone FROM Pessoa AS p
INNER JOIN Telefone as t on p.IdPessoa = t.IdPessoa
WHERE nome LIKE '%' + dbo.Trim(@Nome) + '%')

GO
```

Essa Função irá retornar uma busca na tablea “Pessoa”, onde o nome da pessoa obedecer a condição “WHERE”. Além disso, iremos buscar também os telefones da pessoa, já formatado.

Para executar a Função basta realizar o seguinte comando:

```
SELECT * FROM RetornaPessoa('Maria')
```

Assim, iremos retornar todas as pessoas do nosso banco que possuam “Maria” em alguma parte do nome.

Multi-Statement Function

```
CREATE FUNCTION TipoFuncao ( @CODIGO INT)
RETURNS @TAB_RET TABLE ( COD INT , NOME VARCHAR(20) )
AS
    BEGIN
        IF @Codigo = 1
            BEGIN
                INSERT INTO @TAB_RET VALUES (@CODIGO, 'Retorno
com 1')
            END
        IF @Codigo = 2
            BEGIN
                INSERT INTO @TAB_RET VALUES (@CODIGO, 'Retorno
com 2')
            END
        RETURN
    END
```

Essa função recebe como parâmetro um valor – “Codigo” – e retorna um valor de acordo com a entrada. Caso o valor informado seja “1” ele retornará um resultado. Caso seja “2” retornará outro resultado.

Para executar a Função basta realizar o seguinte comando:

```
SELECT * FROM TipoFuncao(1)
```

ou então

```
SELECT * FROM TipoFuncao(2)
```

Assim, de acordo com o valor passado, a função irá retornar um valor.

Com isso finalizamos a parte de programação em banco de dados. Vimos a importância das Triggers, Stored Procedures, Cursores e Funções, e como

elas são úteis e facilitam o tratamento, atualização e manutenção em um banco de dados.

Linguagem de controle de dados

O SQL possui comandos para permitir ou negar privilégios, variando de cada versão SQL.

- GRANT: autoriza o usuário a executar ou setar operações;
- REVOKE: remove ou restringe a capacidade de um usuário executar operações.

Para relizar testes, vamos criar um LOGIN.

```
CREATE LOGIN ManutencaoDados WITH PASSWORD = 'manutencao'
```

Depois vamos crar um banco de dados para testes e criar uma tabela:

```
CREATE DATABASE Privilegios
GO

CREATE TABLE Teste
(
    [IdTeste] INTEGER IDENTITY (1,1) PRIMARY KEY,
    [DescricaoTeste] VARCHAR (100) NOT NULL
)
```

Agora vamos associar um usuario ao login:

```
CREATE USER [JoseCorrea] FOR LOGIN [ManutencaoDados]
```

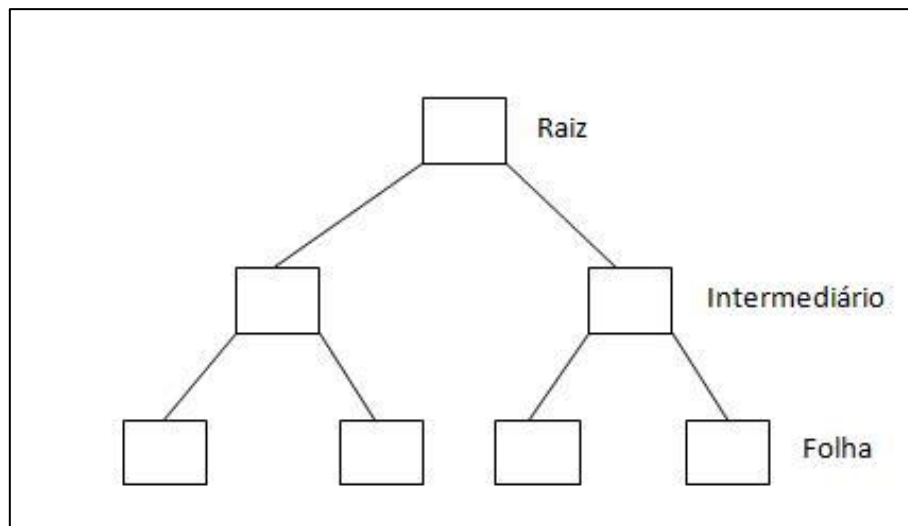
Vamos definir agora, alumas permitir e negar alguns privilégios para o nosso LOGIN

Índice

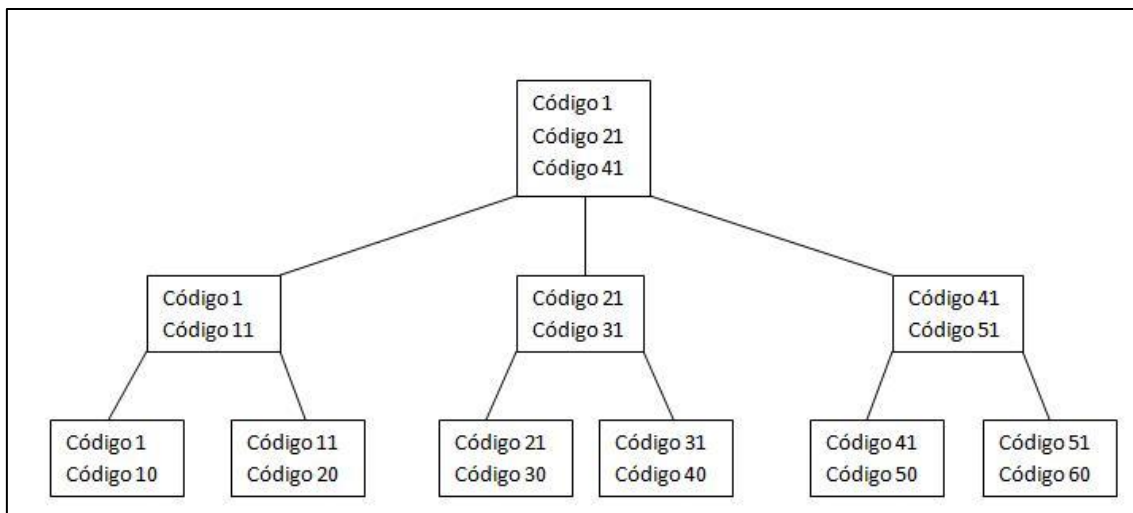
Índices são utilizados para facilitar a busca de informações em uma tabela do banco de dados tentando minimizar a quantidade de operações realizadas para retornar as informações de maneira mais eficiente.

O SQL Server utiliza o modelo *B-Tree* para gravar a estrutura de índice.

- Contém um nó raiz que possui uma única página de dados e subníveis desse nó, chamados de níveis folhas;
- Semelhante a estrutura de árvores em algoritmos;
- Uma *B-Tree* é sempre simétrica, ou seja, possui o mesmo número de páginas a esquerda e a direita de cada nível.



Para ilustrar melhor a utilização da estrutura *B-Tree*, vamos analisar a imagem da busca de um código.



A busca pelo índice é realizada na seguinte sequência:

1. Inicia-se pelo nó raiz;
2. É percorrido todas as linhas até encontrar em qual grupo de valores o item a ser procurado se encontra;
3. Para cada subnível será realizado os passos anteriores até que a operação seja finalizada.

Por exemplo, se quisermos buscar o Código 23, a busca será feita da seguinte maneira:

1. O nível raiz é percorrido;
2. SQL Server identifica que o Código 23 está entre o subnível Código 21 e Código 31 (subnível do meio da subestrutura);
3. Verifica após isso que o Código 23 está no subnível da Esquerda, ou seja, Código 21 e Código 30.

Assim, é possível observar o potencial da criação de um índice para consultas. Nesse exemplo ele conseguiu eliminar mais de 50% das informações em que se poderiam ser buscadas.

Existem diversos tipos de índices que podem ser gerados. Abaixo segue uma descrição resumida de cada índice.

Índices Bons ☺

- Chaves Primárias;
- Chaves Estrangeiras;
- Colunas acessadas por range (*between*);
- Campos utilizando em *group by* ou *order by*.

Índices Ruins ☹

- Campos do tipo *text*, *image*, *decimal*;
- Campos calculados;
- Campos com alta cardinalidade (Masculino ou Feminino).

Índice *Clustered*

- Gerado automaticamente após criar uma chave primária, mas este não está diretamente ligado a chave;
- Ordena os registros por sequência, o que facilita a busca;
- Cada tabela pode possuir apenas um índice *Clustered*.

A sintaxe para criação de um índice *Clustered* é:

```
CREATE CLUSTERED INDEX IX_UF ON UF (UF)
```

Assim, estamos criando um índice com o nome de “IX_UF” para o nosso campo “UF” da tabela “UF”. Para criação desse tipo de índice devemos ter alguns parâmetros pré-estabelecidos:

- Será utilizado como base pelos outros índices;
- É interessante um índice desse tipo quando as chances de repetição do mesmo valor sejam praticamente nulas.

Índice *Non-Clustered*

- Não afetam a forma de armazenamento dos dados;
- É possível mais de um índice *Non-Clustered* em uma tabela. Até 249 índices por tabela;
- Caso exista um índice *Clustered* na tabela, os índices *Non-Clustered* utilizarão ele como referência;
 - Segundo nível de busca(*Clustered* > *Non-Clustered*).

A sintaxe para criação de um índice *Non-Clustered* é:

```
CREATE NONCLUSTERED INDEX IX_Cidade ON Cidade (NomeCidade)
```

Assim, estamos criando um índice com o nome de “IX_Cidade” para o nosso campo “NomeCidade” da tabela “Cidade”.

Os demais índices abaixo serão apresentados somente à nível de curiosidade, onde não especificaremos sintaxes.

Índice *Unique*

- Usados quando os dados indexados não se repetem;
- Uma chave primária, por exemplo, é uma boa candidata a receber um índice do tipo *Unique*.

Índice *Full-Text*

- Utilizados para BLOB's (*Binary Large Objects*);
- Campos de texto são BLOB's;
- Serviço administrado pela ferramenta *Microsoft Full-Text Engine for SQL SERVER*.

Índice *XML*

- Utilizados para BLOB's (*Binary Large Objects*);
- Aplicado para o tipo XML, que pode armazenar até 2GB de dados;
- Custo alto de processamento/manutenção.

Tipos de Índices

Os índices podem ser separados por tipos, e, basicamente, temos dois tipos: índices simples e índices compostos.

Índices simples

- Utilizam apenas uma coluna;
- Normalmente são as chaves primárias das tabelas;

A sintaxe de um índice simples já vimos acima:

```
CREATE NONCLUSTERED INDEX IX_Cidade ON Cidade (NomeCidade)
```

Índices compostos

- Utilizam mais de uma coluna;
- Criados por ordem de seletividade: sequência com os campos que serão definidos no índice.

A sintaxe para criação de um índice composto é:

```
CREATE NONCLUSTERED INDEX IX_Cidade ON Cidade (IdUF, NomeCidade)
```

Assim estamos criando um índice que buscará a informação por dois campos, ou seja, “IdUF” e “NomeCidade”.

A sintaxe geral para criação de um índice é:

```
CREATE [UNIQUE] {CLUSTERED | NONCLUSTERED} INDEX nome
[WITH
[FILLFACTOR = n]
[[,] IGNORE_DUP_KEY]
[[,] {SORTED_DATA | SORTED_DATA_REORG}]
[[,] {IGNORE_DUP_ROW | ALLOW_DUP_ROW}]]
[ON SEGMENT nome_do_segmento]
```

Custo de Índice

Um índice realiza criação de novos objetos e consome recursos do banco de dados e da máquina. Assim, podemos considerar os seguintes critérios de custo ao criar um índice:

- Capacidade de Armazenamento: ao se criar um índice estamos realizando criação de novos objetos no banco. Quanto maior a tabela, maior será a quantidade de índices criados. É possível existir tabelas onde a estrutura de um índice é maior que a estrutura para o armazenamento da tabela;
- Custo de processamento: operações de INSERT, UPDATE e DELETE afetam as páginas de índices, o que interfere na performance da execução.

FillFactor

- Recurso útil para minimizar o problema de manutenção dos índices;
- Especifica a porcentagem de preenchimento que será considerada na construção das páginas de índice:
 - *Fillfactor* de 60% define que vamos preencher apenas esta proporção do espaço físico disponível em cada página, deixando 40% de espaço vazio. Este espaço será usado quando formos incluir novas informações naquela página.
- É um argumento opcional. Seu valor varia entre 0 e 100:

- Se ele for omitido, a página de índice será preenchida o máximo possível, deixando espaço para inserção de mais um único registro.
- O *Fillfactor* ideal pode mudar conforme o tipo de uso do banco de dados:
 - Bancos de dados que registram informações transacionais (OLTP) geralmente usam índices com *Fillfactor* de 70% ou menos. Consideramos este patamar porque sistemas OLTP têm muitas operações de INSERT/DELETE/UPDATE;
 - No caso de sistemas de apoio à decisão (OLAP), os dados são inseridos periodicamente e, geralmente, em lotes de milhares de registros. Nestes casos, os índices usam *Fillfactor* na ordem de 80% a 90%.

Dicas para criação de índices

- Que sejam campos numéricos. Indexar campos alfanuméricos é sempre um problema. As informações costumam consumir tanto espaço nas páginas de índice que o SGBD acaba precisando armazenar um número enorme de páginas. Isto torna o índice menos eficiente;
- Que os campos tenham uma alta seletividade. Um exemplo clássico: para que indexar um campo de Sexo, se só temos as opções Masculino ou Feminino? A seletividade é baixíssima. Ao escolhermos uma das opções, estamos selecionando aproximadamente metade dos registros de toda a tabela;
- Evitar índices compostos sempre que possível. Quando eles realmente forem necessários, manter o índice o mais “curto” possível, ou seja, com o mínimo de colunas. Quanto menor o índice, menos páginas de índice serão necessárias e menos tempo de processamento será necessário para localizar um registro desejado;

- Ainda no caso de índices compostos, observar qual seria a melhor sequência de campos, de modo a otimizar performance;
- Indexar apenas os campos que tragam um alto resultado. Um índice bem projetado pode reduzir por um fator de 5 ou 10 o tempo de resposta de uma consulta. Nunca crie índices para reduzir tempos de resposta a algo que seja superior à metade do resultado da consulta sem índice;
- Se uma tabela merece ao menos um índice, que seja um índice *Clustered*;
- Toda tabela com alguns milhares de registros merece um índice ou, no caso, uma chave primária;
- Em muitos casos, é interessante criar índices *Non-Clustered* em campos que contêm chaves estrangeiras. Isso porque estes campos sempre serão usados em consultas em que fazemos a junção de tabelas.

Otimização de consultas

Já vimos que a utilização de índices pode nos ajudar a melhorar a performance do banco de dados. Mas, como medir isso? Como saber se a criação de um índice está ou não tendo alguma alteração (positiva ou negativa) em nosso banco de dados?

Felizmente existe uma solução! ☺. O PLANO DE EXECUÇÃO! \o/.

A ideia do Plano de execução é apresentar os passos realizados pela operação para se chegar ao resultado esperado.

Consultores de banco de dados utilizam essa ferramenta para verificar a custo de operações em um banco e analisar quais as medidas viáveis podem ser tomadas para que a performance do banco de dados possa aumentar.

Para ilustrar isso, vamos verificar a consulta abaixo, de um banco de dados com muitos registros;

Por curiosidade, vamos verificar a quantidade de registros existentes na tabela “TB_Cobranca”:

Executando o comando:

```
SELECT COUNT(*) FROM TB_Cobranca
```

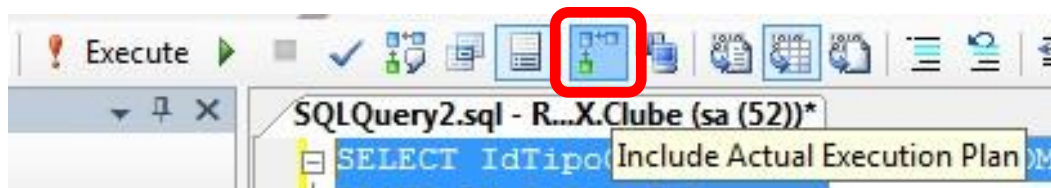
Temos que existem 10.724.980 registros na tabela, ou seja, uma quantidade de registros interessante que possivelmente a utilização de índice pode melhorar o desempenho das operações realizadas no banco.

Vamos executar o seguinte comando:

```
SELECT IdTipoCobranca, COUNT(*) FROM TB_Cobranca  
GROUP BY IdTipoCobranca
```

Com esse comando estamos listando todas as cobranças existentes na tabela “TB_Cobranca” agrupadas pelo campo “IdTipoCobranca”.

O plano de ação pode ser acionado no seguinte local:



Após executar o comando SELECT com essa opção acionada, será criada mais uma aba no retorno da consulta, como a imagem abaixo:

A screenshot of the SQL Server Enterprise Manager interface showing the results of the query. The 'Results' tab is active, displaying a table with three columns: 'IdTipoCobranca', '(No column name)', and a third column with numerical values. The first row is highlighted. The 'Execution plan' tab is also visible and highlighted with a red rectangular box.

	IdTipoCobranca	(No column name)	
1	1	2724192	
2	2	1600158	
3	3	1600158	
4	4	1600157	
5	5	1600157	
6	6	1600158	

Ao clicar nela, podemos visualizar o plano de execução da consulta.

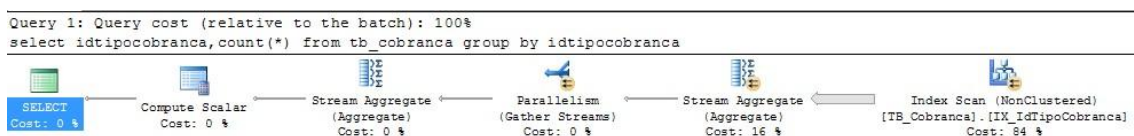


Após isso vamos criar o seguinte índice para tentar facilitar a busca e minimizar recursos. Como o Campo “IdTipoCobranca” é um bom candidato para possuir um índice de consulta, vamos criar um índice com esse valor:

```
CREATE NONCLUSTERED INDEX IX_IdTipoCobranca ON TB_Cobranca
(IdTipoCobranca)
```

Para facilitar a visualização das figuras abaixo, utilize o zoom do leitor do arquivo para que fique mais fácil a leitura das mesmas.

Após a criação desse índice, devemos executar a consulta novamente, agora, com um novo índice. Abaixo o plano de execução da operação com o novo índice:



Podemos ver que a consulta utiliza o novo índice criado. Para a análise utilizarei a comparação de apenas um operador da consulta, comparado o *Clustered Index Scan* (busca pela chave primária) com o *Index Scan* (novo índice criado). A imagem da esquerda é a consulta sem o índice e a da direita a consulta executada com o novo índice criado.

Clustered Index Scan (Clustered)	
Scanning a clustered index, entirely or only a range.	
Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Actual Number of Rows	10724980
Estimated I/O Cost	77,1335
Estimated CPU Cost	5,89882
Number of Executions	4
Estimated Number of Executions	1
Estimated Operator Cost	83,0323 (75%)
Estimated Subtree Cost	83,0323
Estimated Number of Rows	10725000
Estimated Row Size	11 B
Actual Rebinds	0
Actual Rewinds	0
Ordered	False
Node ID	6
Object	
[Clube].[dbo].[TB_Cobranca].[PK_TB_Cobranca_7E6CC920]	
Output List	
[Clube].[dbo].[TB_Cobranca].IdTipoCobranca	

Index Scan (NonClustered)	
Scan a nonclustered index, entirely or only a range.	
Physical Operation	Index Scan
Logical Operation	Index Scan
Actual Number of Rows	10724980
Estimated I/O Cost	10,7987
Estimated CPU Cost	5,89882
Estimated Number of Executions	1
Number of Executions	4
Estimated Operator Cost	16,6975 (84%)
Estimated Subtree Cost	16,6975
Estimated Number of Rows	10725000
Estimated Row Size	11 B
Actual Rebinds	0
Actual Rewinds	0
Ordered	True
Node ID	5
Object	
[Clube].[dbo].[TB_Cobranca].[IX_IdTipoCobranca]	
Output List	
[Clube].[dbo].[TB_Cobranca].IdTipoCobranca	

Análise de índice da consulta

É possível notar que a fonte da busca das informações mudou. Ao invés de realizar a busca pelo objeto “PK_TB_Cobranca_7E6CC920”, o SQL optou por utilizar o novo índice, “IX_IdTipoCobranca”. O Custo estimado de Entradas e Saídas “Estimated I/O Cost” baixou de 77 para 11 (valores arredondados). O custo do CPU “Estimated CPU Cost” se manteve constante. O custo do Operador “Estimated Operator Cost” baixou de 83 para 17, o que foi um ganho muito grande.

Assim podemos dizer que a criação do índice favoreceu a busca de dados, resultado em uma melhoria significativa nos recursos utilizados para se obter uma resposta do banco de dados.

Data Warehouse

Os SPT (Sistema de Processamento de Transações) de um ambiente corporativo lidam com informações operacionais da empresa. A maior responsabilidade deste tipo de sistema é exercer controle dos processos executados no nível operacional. A obtenção de dados variados e diversos sobre as transações é necessária para que esta condição de controle seja satisfeita. Estes dados acabam armazenados em diferentes sistemas e fontes de dados.

Não obstante, as informações operacionais fornecem parâmetros para que gestores de negócio avaliem o desempenho da organização e façam projeções. Mas apenas parte dos dados transacionais é relevante para o nível estratégico. Ainda assim, os SPT não atendem completamente à demanda dos administradores no que diz respeito a suporte a decisões.

Uma solução para a questão é a criação do *Data Warehouse*, uma base de dados com capacidade de integrar informações relevantes para a cúpula organizacional de maneira concisa e confiável. Estas informações se encontram espalhadas nos sistemas de processamento de transações e em fontes externas. O DW se sobressai por fornecer dados de caráter histórico e estatístico aos SAD (Sistema de Apoio à Decisão). Estas duas características dos dados do DW são atributos essenciais para se identificar indicadores, que apresentam o grau de evolução da organização por completo, ou por segmentos. Este tipo de informação é útil para amparar administradores na tomada de decisões.

Notoriamente, os sistemas de apoio à decisão e os sistemas de processamento de transações atendem a diferentes níveis organizacionais: estratégico e operacional, respectivamente. Por causa disso, o *Data Warehouse* deve ser criado em um novo ambiente, já que os fins de um SAD e de um SPT são diferentes.

Poupar esses diferentes sistemas de intervenção entre si é uma boa prática porque a manipulação de dados ocorre de modo distinto entre eles. No

ambiente operacional, os sistemas são do tipo OLTP (*On-Line Transaction Processing* – Processamento Transacional *On-Line*). Já no ambiente estratégico, os sistemas são OLAP (*On-Line Analytical Processing* – Processamento Analítico *On-Line*).

O sistema OLTP armazena as transações de um negócio em tempo real, recebendo dados constantemente. A estrutura de dados deste sistema tem que estar otimizada para validação a entrada, rejeitando dados que não atendem a determinadas regras de negócio. Devido à grande abrangência de sua estrutura de dados, um sistema OLTP fica sobrecarregado quando precisa responder uma consulta de alta complexidade. Este exemplo de consulta acontece quando se tenta levantar informações de caráter estratégico em um sistema OLTP.

No sistema OLAP, a visão é orientada à análise. Assim, a estrutura de dados é montada para que consultas feitas por usuários do nível estratégico da organização retornem resultados em curto tempo. Além disso, as informações obtidas têm propósito exclusivamente analítico.

Características do *Data Warehouse*

O *Data Warehouse* é sustentado por quatro aspectos: orientação por assunto, variação de tempo, não volatilidade e integração.

- Orientação por assunto: as informações armazenadas pelo *Data Warehouse* são agrupadas por assuntos principais de cada uma das áreas essenciais da organização.
- Variação de tempo: diferente de um banco de dados transacional, que armazena valor corrente, o *Data Warehouse* guarda dados históricos. Como é indispensável que o dado do DW refira-se ao seu momento de inserção no sistema transacional, a data lhe é um elemento chave.
- Não volatilidade: o *Data Warehouse* é um ambiente exclusivamente de consulta. Isto impede que seus dados sejam editados. Depois que

determinada informação relacionada do ambiente transacional é inserida no DW, as únicas operações que Podem ser aplicadas a ela são (i) acesso, sem necessidade de bloqueio por concorrência de usuários, e (ii) exclusão, caso seja decidido que um dado não deve mais fazer parte do DW.

Integração: como os dados inseridos no *Data Warehouse* provém de um ambiente de múltiplas plataformas sistêmicas, é necessária a unicidade de informações. Para alcançar isso, o DW segue convenção de nomes e valores de variáveis, seguindo um padrão para um mesmo tipo de elemento oriundo de plataformas distintas.

Componentes do *Data Warehouse*

Numa visão abrangente, é possível separar o *Data Warehouse* em três partes elementares expostas a seguir.

1. Papéis

Este componente envolve as funções e responsabilidades presentes no desenvolvimento do *Data Warehouse*. Devido ao fato do DW abranger variados tipos de desenvolvedores e usuários, o cuidado com a gestão de papéis neste ambiente é maior do que a atenção dada ao mesmo assunto nos sistemas operativos.

Os papéis presentes em um *Data Warehouse* são:

- Analistas responsáveis pela carga dos dados: conhecem e trabalham o mapeamento do DW com o banco de dados do sistema operativo e os requisitos de filtragem e integração;
- Usuários finais: são os gerentes, executivos e analistas do negócio que tomam decisões. Eles conhecem os problemas e oportunidades da empresa. Podem ser usuários diretos, aqueles que acessam

informações de todo o DW, ou indiretos, os que acessam informações de partes do DW;

- Analistas responsáveis pelo desenvolvimento e manutenção do DW: são os DBA (*Database Administrator* – Administrador de Banco de Dados) e os DA (*Data Administrator* – Administrador de Dados) dos SGBD (Sistema Gerenciador de Banco de Dados) dos sistemas operativos. Cuidam dos metadados, da arquitetura e da estrutura dos dados, visando o bom desempenho de consultas.

2. Processos e Ferramentas

No *Data Warehouse*, processos se resumem à extração dos dados dos sistemas operativos, na organização e integração destes dados de forma consistente para o DW e no acesso aos dados para consulta. Uma questão importante que deve ser observada no desenvolvimento destas atividades é a garantia de consistência e integridade das informações para que elas retratem a realidade de negócios da organização.

Para a realização dos processos são utilizadas ferramentas que auxiliam na execução das atividades. Dentre estas ferramentas estão aplicações que ficam responsáveis por filtrar, limpar, sumarizar e concentrar os dados de fontes externas e de sistemas operativos. Além destas aplicações, há também as responsáveis pelo acesso aos dados, que servem para permitir um acesso intuitivo aos dados, possibilitando a análise de informações mais significativas.

Geralmente, o *Data Warehouse* utiliza os seguintes tipos de ferramentas:

- Ferramentas para pesquisa e relatórios: oferecem interface gráfica para geração de relatórios e análise de dados históricos;
- Ferramentas OLAP: proporciona uma melhor visão analítica para identificar de maneira mais fácil as causas de resultados obtidos. São divididas em:

- ROLAP (*Relational On-Line Analytical Processing* – Processamento Analítico On-Line Relacional): ferramentas OLAP que acessam banco de dados relacionais;
- MOLAP (*Multidimensional On-Line Analytical Processing* – Processamento Analítico On-Line Multidimensional): ferramentas OLAP que acessam banco de dados multidimensionais;
- HOLAP (*Hybrid On-Line Analytical Processing* – Processamento Analítico On-Line Híbrido): ferramentas OLAP que acessam tanto banco de dados relacionais quanto banco de dados multidimensionais;
- DOLAP (*Desktop On Line Analytical Processing* – Processamento Analítico On-Line em Desktop): ferramentas OLAP que acessam banco de dados de computadores pessoais;
- SIE (Sistema de Informações Executivas): apresentam uma visualização mais simplificada dos dados, com informações mais consolidadas, não requerendo experiência e tempo do usuário para fazer análise;
- *Data Mining*: ferramentas que permitem que o usuário avalie tendências e padrões não visualizáveis nos dados.

3. Dados

Dados são armazenados no *Data Warehouse* em níveis de agregação diferentes dos sistemas operativos. Enquanto que nos sistemas operativos os dados são detalhados, no DW os dados são levemente ou altamente sumarizados. Para que o trabalhoso processo de extração e tratamento de dados do sistema operativo não seja em vão, os repositórios de dados devem estar prontos para receber corretamente os dados, respeitando as características de construção do DW. A implementação de cada repositório depende da arquitetura de dados apresentada pela empresa;

Em suma, quatro tipos de repositórios são comuns no *Data Warehouse*:

- ODS (*Operational Data Storage* – Armazenamento de Dados Operacionais): repositório de armazenamento intermediário dos dados. Seus dados ficam em um nível de compatibilidade com os dados dos sistemas legados. Por isso, o ODS é uma base atual, ou quase atual. Isso faz com que seja usada (i) como uma área provisória de reorganização física de dados operacionais extraídos, (ii) para fornecer relatórios operacionais, (iii) dar suporte a decisões operacionais e (iv) ponto de consolidação, caso os dados operacionais sejam de várias fontes. O principal amparo fornecido pela utilização do ODS no DW é o fato de que o ODS ajuda a evitar o carregamento indevido no DW;
- *Data Warehouse*: repositório com grande capacidade de armazenamento, que integra os principais dados gerenciais da organização de maneira concisa e confiável. É a base de dados usada pelos SAD. Armazena informações de cunho histórico e estatístico;
- *Data Mart*: subconjunto de dados do *Data Warehouse*. O DM possui dados direcionados a uma área específica de negócio e permite acesso de dados de modo descentralizado;
- Cubo: banco de dados com escopo de informações reduzido e processamento acelerado. Permite ao usuário armazenar dados em caráter temporário e ter uma visão analítica rápida por ser manipulado através ferramentas OLAP.

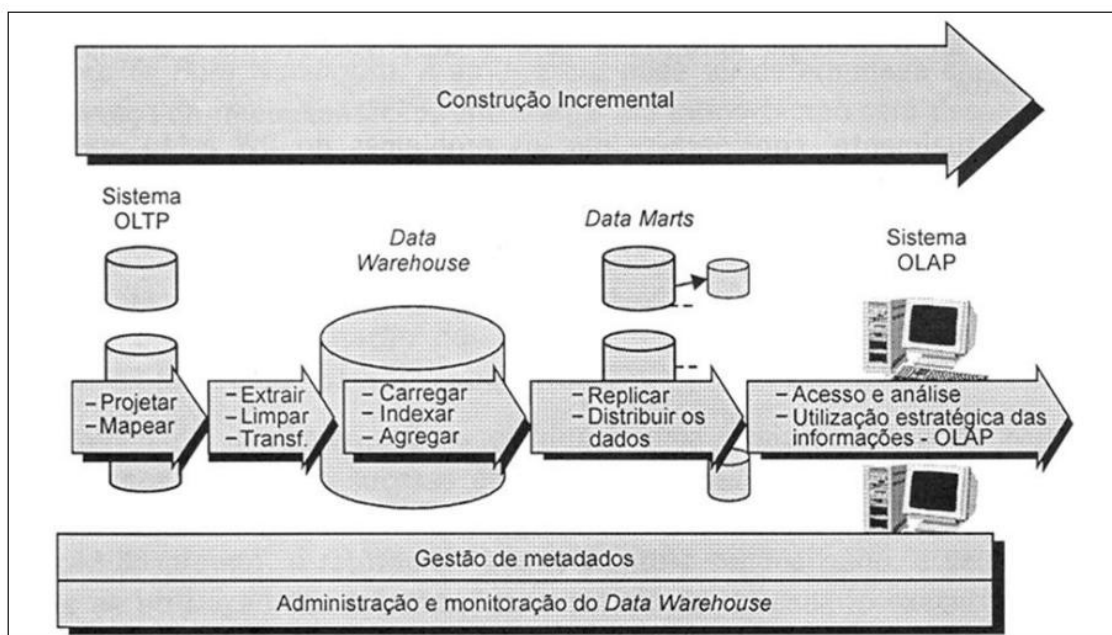
Tecnologia de *Data Warehousing*

Como o *Data Warehouse* tem o objetivo fornecer informações de apoio à tomada de decisões, sua construção deve ser guiada por necessidades apontadas pelos gestores do negócio. Em cima destas condições, o DW é projetado e construído.

O processo de se fazer *Data Warehouse* acontece através da tecnologia de *Data Warehousing*. Esta técnica envolve a criação do projeto de *Data Warehouse*, passa pela implementação do mesmo e até chegar à sua utilização pelos executivos da empresa.

Os desenvolvedores de *Data Warehouse* têm que buscar a melhor alternativa de integrar o DW às diferentes fontes de dados existentes. Isto envolve a escolha da arquitetura e implementação adequadas para suprir as necessidades da organização.

Porém, apenas estes cuidados não tornam certo o sucesso do *Data Warehouse*. O *Data Warehousing* é um processo incremental, e uma constante administração e monitoração do *Data Warehouse* garante que ele ampare eventuais mudanças estratégicas e estruturais da empresa.



A construção do *Data Warehouse* é feita por etapas que são interdependentes e incrementais. Assim, devem ser bem administradas e monitoradas sempre. Isto é ilustrado na figura acima. Conforme apresentado na figura, primeiro, o DW deve ser projetado e mapeado com base num sistema OLTP. Somente depois desta análise, os dados são retirados do sistema transacional, tratados e carregados para o *Data Warehouse*, onde ficam distribuídos e ou replicados nos Data Marts. Depois

de toda esta montagem, o DW fica com acesso disponível para análise e utilização estratégica no sistema OLAP.

ETL (*Extract, Transform and Load* – Extração, Transformação e Carga)

Dentre as atividades a serem feitas no *Data Warehousing* estão a extração, a transformação e a carga dos dados. ETL é um processo crítico e complexo, pois os dados que compõem o *Data Warehouse* são descendentes de diferentes sistemas OLTP. Isso faz com que seja necessário definir quais informações levar ao DW e realizar adaptações relevantes como padronizações de codificação, formato e unidades de medida para um mesmo tipo de dado que está inserido de diferentes formas nos sistemas legados.

O processo de extração, transformação e carga de dados envolve várias operações, cada uma contendo suas considerações especiais:

- **Extração:** processo de captura dos dados de banco de dados operacionais e outras fontes. Tende a ser um processo muito intenso em termos de entrada e saída, e que, para evitar interferência em outras operações do sistema de origem, normalmente é realizada em paralelo;
- **Limpeza:** aprimoramento dos dados antes deles serem inseridos no DW. Geralmente é feito em lotes e envolvem operações como preenchimento de valores omitidos, correção de erros de digitação e outros erros de entrada de dados, estabelecimento de abreviações e formatos padronizados, substituição de sinônimos por identificadores padrão, entre outras. Os dados que não podem ser esmerados neste processo são depostos;
- **Transformação e consolidação:** modificação e mescla que precisam ser feitas nos dados e entre eles, para que sejam inseridos de forma correta no DW. Nesta fase, faz-se a divisão e ou a combinação de

registros retirados das tabelas do esquema físico de um ou mais bancos de dados operacionais;

- **Carga:** transporte de dados transformados e consolidados para o DW, fazendo a verificação de integridade e construindo quaisquer índices necessários;
- **Renovação:** atualização periódica dos dados do DW. Pode ser feita de modo parcial ou completo e envolve os mesmos pontos associados à operação de carga.

Arquitetura de *Data Warehouse*

Como o projeto de *Data Warehouse* envolve um alto nível de complexidade, a definição e a construção da estrutura que comportará o DW é um ponto relevante na fabricação desta base de dados.

A escolha da arquitetura do DW, geralmente, considera fatores como (i) infraestrutura disponível, (ii) porte da organização, (iii) abrangência do projeto, (iv) capacitação dos empregados da empresa, e (v) recursos de investimento.

Analisando estes componentes evita-se a perda da característica de sinergia que deve existir entre o *Data Warehouse* e a estratégia organizacional.

Tipos de Arquitetura

Existem três tipos de arquitetura de Data Warehouse: global, independente e integrada.

Arquitetura Global

Nesta arquitetura, o *Data Warehouse* é projetado e construído para atender às necessidades da empresa como um todo. Isto faz com que cada

departamento da empresa tenha um alto grau de acesso às informações, e que estas sejam muito utilizadas. Os usuários finais utilizam visões corporativas de dados.

O processo de extração, transformação e carga dos dados no *Data Warehouse* de arquitetura global deve ser feito fora do horário de pico de operações da organização. Caso contrário, os procedimentos no ambiente transacional, que é a fonte dos dados que serão levados ao DW, podem ser prejudicados. Os dados dos sistemas operativos e de eventuais fontes externas são extraídos em lote, em seguida filtrados e transformados de acordo com as necessidades levantadas no projeto de DW antes de serem carregados para o repositório.

Um ponto relevante no que diz respeito à arquitetura global é que ela se refere ao escopo de acesso e utilização das informações na empresa, e não a uma centralização física da base de dados. A arquitetura global pode ser fisicamente centralizada ou fisicamente distribuída.

Quando a empresa está estabelecida em apenas um local, existe uma centralização física do *Data Warehouse*. Já se a empresa está instalada em diferentes locais, os dados podem estar fisicamente distribuídos em vários repositórios repartidos nos diferentes estabelecimentos da empresa.

Arquitetura Independente

Quando o *Data Mart* é feito para atender às necessidades de um único departamento da empresa, sem nenhum foco corporativo, têm-se uma arquitetura independente de *Data Warehouse*. Nesta arquitetura, um DM não tem conectividade com outro DM de um setor diferente da organização.

Se uma informação de outro *Data Mart* for relevante a um *Data Mart* de uma arquitetura independente, estas informações são importadas e ajustadas.

A vantagem deste tipo de arquitetura é a rapidez de implementação. Porém, não é possível ter uma visão total da empresa porque sua restrição possui um mínimo de integração corporativa. Geralmente, isto também torna o *Data Mart* inacessível a outros departamentos, se não o que possui o repositório de dados.

Arquitetura Integrada

A mescla das duas arquiteturas anteriores resulta na arquitetura integrada de *Data Warehouse*. Os *Data Marts* são implementados em cada departamento da empresa e, posteriormente, integrados ou interconectados. Além de cada departamento possuir um *Data Mart*, como numa arquitetura independente, há uma visão corporativa maior das informações, similar à arquitetura global.

O nível de complexidade da arquitetura integrada é considerável, pois o requisito de conectividade presente nela demanda funções e capacidades maiores do que as encontradas na arquitetura independente. No entanto, a dificuldade em se montar a arquitetura integrada é menor que a de se fazer a arquitetura global, que necessita que toda a estrutura do *Data Warehouse* esteja pronta antes de sua implementação.

Tipos de Implementação do *Data Warehouse*

Assim como a definição da arquitetura do *Data Warehouse*, o tipo de implementação também depende dos recursos disponíveis à construção do DW e dos requisitos que este deve atender. Infraestrutura de Tecnologia da Informação, arquitetura escolhida, escopo da implementação e níveis de acesso são exemplos de fatores que influenciam na opção por um tipo de abordagem de implementação.

As abordagens de implementação de *Data Warehouse* consideradas mais importantes são a (i) *top down*, a (ii) *bottom up* e a (iii) combinação entre a *top down* e a *bottom up*.

Implementação *Top Down*

Dentre as implementações existentes, a *top down* é a que envolve maior tempo para planejamento e trabalho, mas que, em compensação, tem como produto o *Data Warehouse* que respeita totalmente as regras de negócio da organização. Antes de iniciar o projeto de DW, são tratadas as definições conceituais de tecnologia, sendo abordadas questões relevantes estrategicamente e de alcance a todo o corpo da empresa.

Pontos importantes devem ser determinados na fase de planejamento, como fontes de dados que serão utilizadas, estrutura de dados, qualidade de dados a ser considerada e padrões de dados. Para isso, é necessário conhecimento do modelo de dados dos sistemas transacionais.

O processo na implementação *top down* inicia-se com a extração, a transformação e a integração dos dados de sistemas operativos na base de armazenamento intermediária, por exemplo, o ODS. Em seguida, os dados e metadados são transferidos diretamente para o *Data Warehouse*. Somente depois disto, são extraídos os dados e metadados para os *Data Marts*.

Nos DM, o nível de sumarização dos dados é menor do que o existente no DW, além de, geralmente, não apresentarem o nível histórico do DW.

A centralização do repositório de metadados e regras existentes e a existência de uma consequente herança de arquitetura presentes na implementação *top down* permitem a manutenção mais simples do que aquelas realizadas especificamente em cada um de vários repositórios espalhados. Além disso, a concentração de todos os negócios no *Data Warehouse* garantem uma melhor visão de empreendimento da empresa.

Há também desvantagens na implementação *top down*. Como seu desenvolvimento é muito longo e trabalhoso, ele fracassa, caso ele não seja bem planejado e trabalhado. Ainda considerando seu tempo de desenvolvimento, a implementação *top down* pode induzir à frustração de expectativas em toda a organização. Tudo isto gera um alto nível de risco para o investimento neste tipo de ambiente.

Implementação *Bottom Up*

Em contrapartida à implementação *top down*, a implementação *bottom up* proporciona resultados mais rápidos com uma estrutura menos complexa, pelo menos a princípio, e menos dispendiosa. Nesta implementação, os *Data Marts* são desenvolvidos antes do *Data Warehouse*. Este é formado pela junção incremental de *Data Marts* construídos de modo independente.

Desse modo, o *Data Warehouse* é construído sem uma prévia definição de estrutura corporativa. Assim, faz-se necessária uma atenção à metodologia de desenvolvimento para coordenar a elaboração incremental do *Data Warehouse*. Isto ajuda na prevenção da existência de metadados sem padronização e de dados redundantes e inconsistentes.

A brevidade do desenvolvimento *bottom up* gera a maioria dos benefícios deste tipo de implementação. Amostra disso, os *Data Marts* podem ser colocados em produção num prazo relativamente curto. Isto gera maior confiança nas pessoas que compõem a organização, as quais visualizam melhor os resultados. Uma das boas consequências disto é geração de estímulos para investimentos adicionais no projeto.

Outro ponto vantajoso na implementação *bottom up* é o desenvolvimento incremental do *Data Warehouse*. Ele permite que os principais negócios da empresa sejam enfocados inicialmente, sem que haja gastos com áreas menos importantes. Além disso, outro ganho diz respeito ao conhecimento adquirido pela equipe de desenvolvimento dos *Data Marts* que ganha

aprendizado ao término do desenvolvimento de cada *Data Mart*. Isto gera menos riscos nos resultados do *Data Warehouse*.

Já as desvantagens da implementação bottom up estão relacionadas com o risco de perda da visão geral do empreendimento da empresa. Como o desenvolvimento dos *Data Marts* é independente, há chances de inviabilização de integração, que devem ser minimizadas com uma boa coordenação dos desenvolvimentos dos *Data Marts*.

Implementação Combinada

Uma alternativa às implementações *top down* e *bottom up* é a implementação combinada que integra ambas. Neste ambiente, o *Data Warehouse* é modelado numa visão macro e os *Data Marts* são gerados a partir do macro modelo de dados. Em seguida, depois de prontos, os DM são integrados ao modelo físico do DW.

A característica de herança de modelo existente na implementação combinada evita desvantagens significativas das implementações top down e bottom up. Os *Data Marts* gerados têm dados consistentes em virtude do mapeamento e controle dos dados, que é resultado da existência de apenas um modelo de dados. Isto retira a atenção dada ao controle de padronização e consistência de dados de *Data Marts*, atributo da implementação bottom up. Ao mesmo tempo, como os *Data Marts* são construídos separadamente na implementação combinada, as principais áreas de negócios podem ser priorizadas.

Assim, resultados podem ser apresentados à organização de maneira mais rápida e satisfatória. Além disso, a construção evolutiva de vários DM permite aprendizado da equipe de desenvolvimento do DW. Estes dois efeitos da fabricação separada de *Data Marts* contrabalançam com duas desvantagens da implementação *top down*: resultados demorados e ausência de aprendizado da equipe de desenvolvimento.

Em suma, a implementação combinada é a junção o planejamento top down com o desenvolvimento *bottom up*, sendo que as características destas duas partes ficam presentes no *Data Warehouse*.

Modelagem Multidimensional de Dados

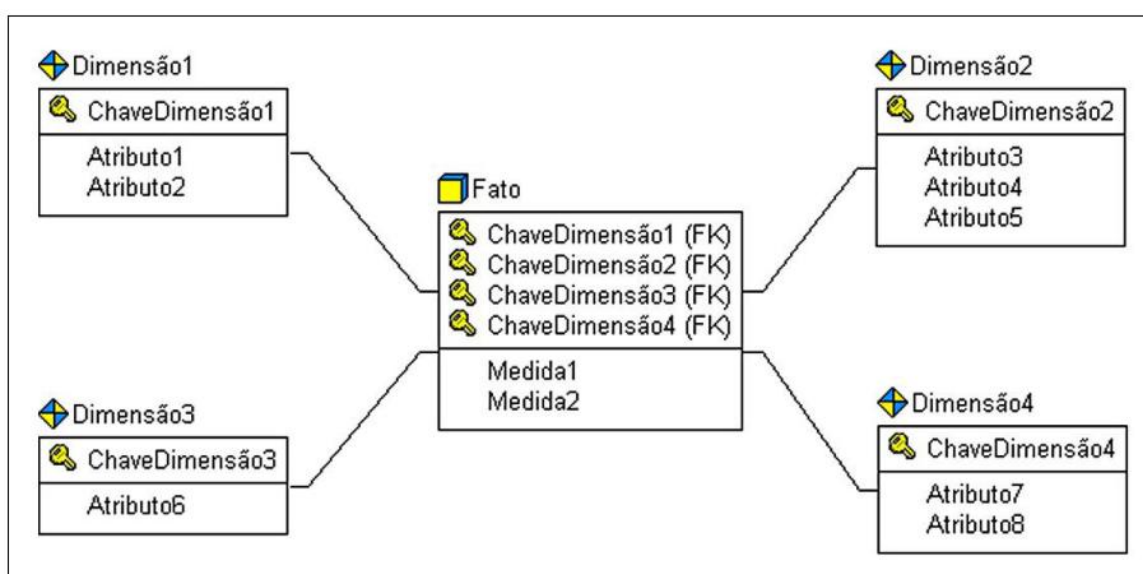
Um aspecto preponderante para se realizar modelagem de dados para *Data Warehouse* é o foco no negócio da empresa. Enquanto que o foco da modelagem de dados para sistemas operativos enfoca o controle de negócios, a modelagem de dados do DW deve permitir que questões abstratas do negócio sejam visualizadas pelos usuários finais, os tomadores de decisão. Esta é uma diferença essencial a ser considerada antes de se desenvolver a modelagem de dados para sistemas de ambiente OLAP. Devido a esses focos distintos, não é apropriado aplicar completamente a teoria relacional, utilizada em sistemas OLTP, na modelagem de dados para *Data Warehouse*.

A base de dados nos sistemas operativos segue as regras de normalização para garantir a consistência dos dados e a diminuição do espaço de armazenamento. Para se realizar algumas consultas numa base de dados deste tipo são necessárias operações complexas e lentas, que fazem junção de várias tabelas do banco de dados. Entretanto, isto é inviável no *Data Warehouse*. Neste, as consultas realizadas pelos usuários finais devem ser de manipulação fácil e retorno de resposta rápido. Para que esta característica seja atendida, o DW, geralmente, tem um modelo de dados sem normalização.

No *Data Warehouse*, a base de dados é construída sob a modelagem multidimensional, que “é uma técnica de concepção e visualização de um modelo de dados de um conjunto de medidas que descrevem aspectos comuns de negócios” (MACHADO, 2006, p. 79). A modelagem multidimensional é “utilizada especialmente para sumarizar e reestruturar

dados e apresentá-los em visões que suportem a análises desses dados” (MACHADO, 2006, p. 79).

O modelo entidade-relacionamento pode ser utilizado para ambiente de *Data Warehouse* com técnica para modelagem multidimensional específica como mostra a figura abaixo. Isto não interfere no suporte ao ambiente de análise multidimensional de dados e ainda facilita a modelagem de dados para desenvolvedores que estão acostumados com a modelagem entidade-relacionamento.



Na figura acima, podem ser identificados os três elementos básicos que formam um modelo multidimensional: (i) fatos, (ii) dimensões e (iii) medidas. O fato é representado no modelo pela tabelas central. Os campos *Medida1* e *Medida2* da ilustração servem para armazenamento de medidas, os valores numéricos que serão analisados. As chaves estrangeiras da tabela fato, que devem fazer parte da composição da chave primária, se relacionam com tabelas dimensões, que são as perspectivas de análise do fato. As dimensões contêm atributos que descrevem a perspectiva de análise.

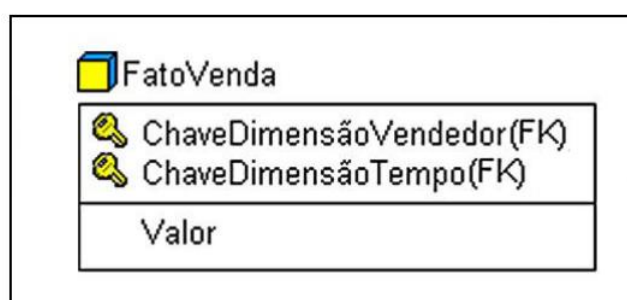
Compreender os conceitos de fatos, dimensões e medidas é importante para se construir modelos multidimensionais. Por isso, estes elementos são conceituados a seguir.

Fato

Os elementos principais de um modelo multidimensional são os fatos. Um fato é todo componente de negócio que pode ser representado por valores numéricos que evoluem no decorrer do tempo e que podem ter históricos mantidos. O *Data Warehouse* nada mais é do que a história de um conjunto de fatos da organização.

A definição dos fatos de um modelo multidimensional requer uma atenção por parte dos analistas responsáveis pelo desenvolvimento do *Data Warehouse* que vai além da abordagem tecnológica. Os fatos devem referenciar os principais componentes de negócio da empresa para que os executivos tenham apoio na análise de negócio de vários pontos de vista.

Assim, as decisões são tomadas com base no comportamento dos fatos.



Por exemplo, a figura acima mostra a tabela *FatoVenda*. A venda está entre os principais componentes de negócio de um estabelecimento comercial. O atributo Valor de *FatoVenda* guarda os valores numéricos que terão seu histórico mantido e servirão para análise. A chave primária da tabela *FatoVenda* é composta por duas chaves estrangeiras, frutos de relacionamentos identificadores com dimensões vendedor e tempo. Estes relacionamentos servem para definir perspectivas de análise do fato venda.

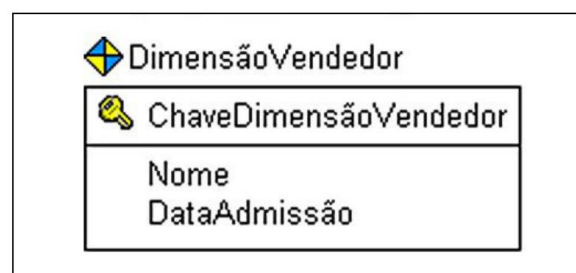
Assim, podem ser feitas análises do valor de vendas por determinado vendedor e ou por determinado período de tempo.

Dimensão

O fato é constituído por dimensões, que são elementos que propiciam, cada um, uma diferente perspectiva de análise do fato. Desse modo, as dimensões determinam o contexto de um assunto de negócio. Por exemplo, o assunto de negócio venda pode ser analisado por vários aspectos de negócios como vendedor e tempo. Neste caso, os aspectos vendedor e tempo são as dimensões do fato venda.

Um caso especial entre as dimensões é a dimensão tempo, que deve estar presente em todos os fatos do *Data Warehouse*. Isto se justifica pelo princípio de que o DW deve armazenar os dados de um assunto de negócio ao longo do tempo. Portanto, a dimensão tempo serve para atender o caráter histórico do *Data Warehouse*.

A representação de uma dimensão no modelo multidimensional é feita através de uma tabela definida por sua chave primária, que mantém a integridade referencial numa tabela fato. Além da chave primária, a dimensão possui atributos de descrição e de classificação. As dimensões normalmente não têm atributos numéricos.



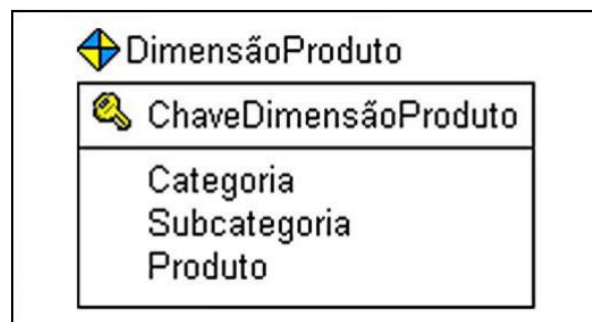
A figura acima mostra a tabela *DimensãoVendedor* que se relaciona com a tabela *FatoVenda* apresentada de exemplo na figura anterior a essa. A chave da dimensão vendedor serve para manter a integridade referencial na

tabela fato e os atributos Nome e *DataAdmissão* descrevem a dimensão vendedor.

Hierarquia de Dimensões

Normalmente, as informações presentes no *Data Warehouse* são hierarquizadas para fins de análise. Isto faz com que seja possível agrupar informações tanto por níveis gerais ou níveis mais específicos.

A visualização de hierarquia nas dimensões é feita através do nível de seus atributos. Por exemplo, um produto pode pertencer a uma subcategoria e esta fazer parte de uma categoria. Assim, estes três atributos se relacionam mantendo uma subordinação onde, hierarquicamente, a categoria pertence ao maior nível, a subcategoria ao nível intermediário e o produto em si ao menor nível. Isto é exibido na tabela *dimensãoProduto* da figura abaixo.

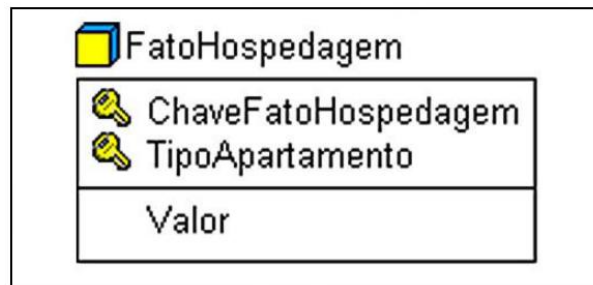


A dimensão pode apresentar uma ou mais hierarquias, juntamente com outros atributos que não pertencem a nenhuma hierarquia.

Dimensão Mascarada

Quando a entidade dimensão não possui um número significativo de ocorrências, ela pode ser implementada como atributo do fato ao qual ela está relacionada. Neste caso, a dimensão é denominada como dimensão mascarada.

A dimensão mascarada continua compondo a chave primária da tabela fato pelo qual é referenciada, assim como aconteceria se ela fosse uma dimensão comum. Isto ocorre porque também será necessário classificar as informações pela dimensão mascarada, e somente com ela estando na chave do fato é que se podem inserir dados distintos para cada ocorrência dela.



Na figura acima, tem-se uma amostra de dimensão mascarada. A tabela *FatoHospedagem* conta com a dimensão mascarada *TipoApartamento*. Os tipos de apartamento oferecidos em hotéis não representam uma variedade significativa. Dependendo do contexto, com apenas as ocorrências simples, duplo e suíte pode-se preencher satisfatoriamente a dimensão tipo de apartamento. A hospedagem também pode ser classificada por tipo de apartamento já que a dimensão mascarada *TipoApartamento* compõe a chave primária da tabela *FatoHospedagem*.

Medida

O fato é qualificado pela medida, um atributo numérico que representa o desempenho de um indicador de negócio relativo um contexto determinado pelas dimensões que participam do fato.

As medidas são classificadas em (i) valores aditivos, (ii) valores não aditivos e (iii) valores semi-aditivos.

- Valores aditivos: números absolutos que podem ser manipuladas algebricamente. Estes valores devem ser identificados para se modelar o *Data Warehouse*, pois compõem os indicadores de desempenho de negócio.

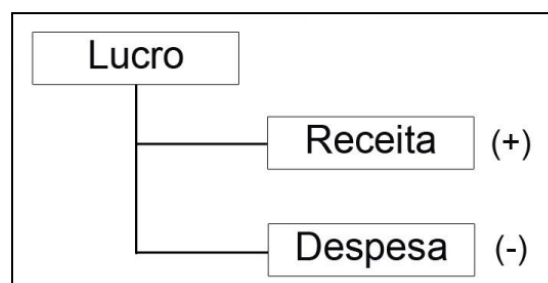
- Valores não-aditivos: valores que não podem ser manipulados livremente. Por exemplo, números relativos, que tem seus valores baseados em números absolutos. Estes valores devem ser identificados para se modelar o *Data Warehouse*, pois, assim como os valores aditivos, compõem os indicadores de desempenho de negócio.
- Valores semi-aditivos: valores que contém contagem dupla. Estes valores podem ser calculados envolvendo valores aditivos ou valores não-aditivos, desde que se refiram exclusivamente a uma determinada medida de base.

Hierarquia de Medidas

A medida pode possuir hierarquia de composição de seu valor. Porém, no modelo multidimensional, a tabela fato pode ter ou não todos os elementos dessa hierarquia.

A hierarquia de medida pode ser visualizada através de uma formulação matemática de forma hierárquica, onde o produto da operação depende dos fatores que a geram.

No exemplo da figura abaixo, o atributo Lucro é composto por dois elementos menores: Receita e Despesa. Tem-se representada uma hierarquia de medidas, onde a subtração do valor de Despesa no valor da Receita gera o atributo Lucro.



Na representação das medidas do fato do modelo multidimensional, a medida Lucro poderá ou não estar acompanhada das medidas Receita e

Despesa, que estão num menor nível hierárquico. O importante é que na composição do valor de lucro, a hierarquia foi respeitada.

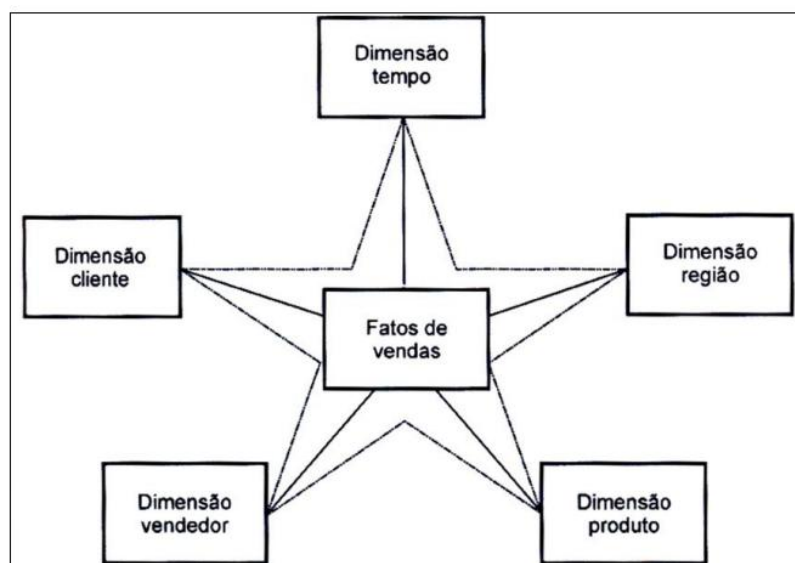
Tipos de Modelo Entidade-Relacionamento Aplicados com Técnica Multidimensional

Existem dois principais tipos de modelo entidade-relacionamento que utilizam técnica de modelagem multidimensional: modelo estrela e modelo floco de neve.

Modelo Estrela

A composição típica do modelo em estrela é representada por uma entidade central, o fato, relacionado a entidades menores, ou seja, as dimensões, que ficam distribuídas ao redor do fato, formando uma estrela. O estilo assimétrico do modelo estrela facilita a leitura e a interpretação.

Esta representação é apresentada na figura abaixo. A entidade central é o fato venda que tem disposta ao seu redor as dimensões região, produto, vendedor, cliente e data.



Os relacionamentos das entidades dimensões com a entidade fato é uma simples ligação entre duas entidades em um relacionamento de um para muitos no sentido da dimensão para o fato. Numa visão entidade-relacionamento tradicional, a entidade fato é associativa.

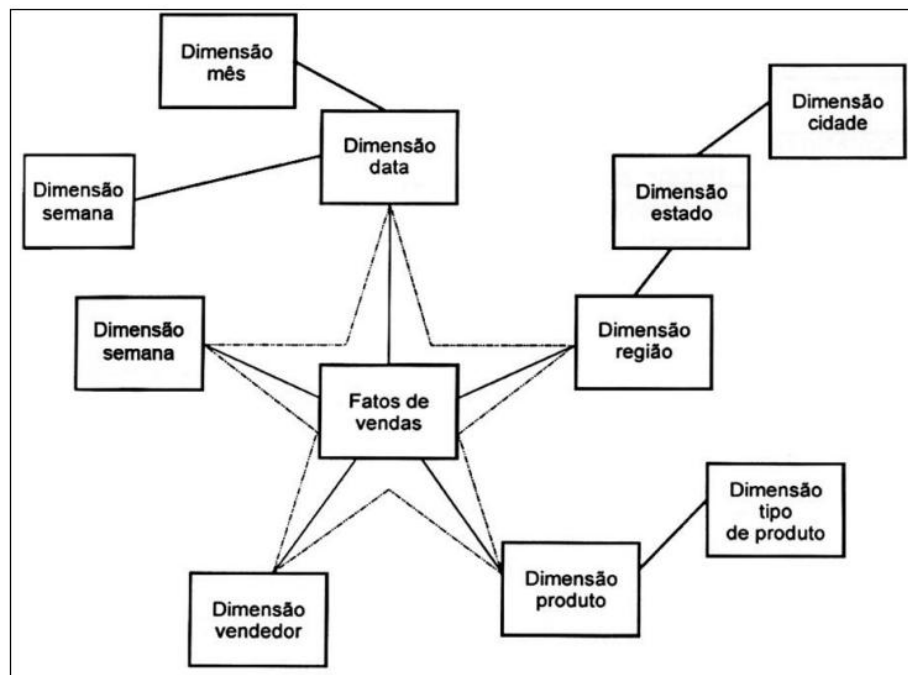
Modelo Floco de Neve

Com a estrutura do modelo estrela como base, o modelo floco de neve é o resultado da decomposição de uma ou mais dimensões que possuem hierarquia entre seus membros.

O modelo floco de neve é uma das maneiras de representar a hierarquia de dimensões. Nele, os atributos de menor nível hierárquico se transformam em outras dimensões que podem se ligar uma nas outras até chegar à dimensão que contém o atributo de maior nível hierárquico, ou com cada atributo se relacionando diretamente com a dimensão que contém o atributo de maior nível hierárquico.

O entendimento deste modelo para desenvolvedores de sistemas OLTP é fácil porque o modelo floco de neve é a aplicação da terceira forma normal sobre as entidades da dimensão. Assim, como num modelo entidade-relacionamento, é evitada a redundância de valores textuais em uma tabela.

Cabe lembrar que o *Data Warehouse* não possui inclusão de dados de digitação, não necessitando normalização do modelo para garantir unicidade de valores textuais. Isto deve ser provido anteriormente nos sistemas legados.



Na figura acima, além da entidade fato venda rodeada pelas dimensões região, produto, vendedor, cliente e data, como no modelo estrela, o modelo floco de neve é caracterizado pela criação das dimensões semana, mês, estado, cidade e tipo de produto, que formam a hierarquia das dimensões às quais estão ligadas. A dimensão tipo de produto é a hierarquia de menor nível da dimensão produto. A dimensão região esta ligada à dimensão estado, nível intermediário da hierarquia, que se relaciona com a dimensão cidade, menor nível da hierarquia. Já na dimensão data existem duas hierarquias que são representadas pelas dimensões mês e semana, onde ambas ligam-se diretamente à dimensão data.

Granularidade

A navegação feita pelos tomadores de decisão nas informações disponibilizadas pelo sistema OLAP começa com a visualização das informações no nível maior de agregação e depois estas informações podem ser detalhadas até o menor nível de sumarização disponibilizado pelo *Data Warehouse*.

Este nível de sumarização dos elementos e de detalhe disponível nos dados é denominado granularidade. O nível de detalhe é inversamente proporcional ao nível de granularidade. Quanto menos detalhado for o dado, maior a granularidade; quanto mais detalhado for o dado, menor a granularidade.

A relevância da granularidade é tanta que ela é considerada o aspecto mais importante do projeto de um *Data Warehouse*. Diferente do que acontece no sistema OLTP, quando é certo que os dados são armazenados no menor nível de granularidade, a granularidade não é um pressuposto no ambiente de um *Data Warehouse*. O volume de dados residentes no DW e o tipo de consulta que pode ser atendido por ele é afetada profundamente pela sua granularidade.

Portanto, a definição do nível de granularidade deve ser analisada e definida antes da construção do modelo multidimensional de dados. Há de se lembrar que o *Data Warehouse* deve guardar informações que auxiliem a tomada de decisão com uma velocidade de consulta maior que a de sistemas OLTP e, além disso, manter a historicidade dos dados.

Desse modo, os requisitos que devem ser atendidos para a definição da granularidade de um dado é estabelecer até que nível o dado deve ser detalhado para dar suporte à tomada de decisão e se o volume de dados gerados com isso será capaz de prejudicar o desempenho do sistema.

Operações OLAP

A funcionalidade de ferramentas OLAP é caracterizada pela análise multidimensional dos dados, apoiando o usuário final ao extrair dados do *Data Warehouse* e construir relatórios capazes de responder questões gerenciais.

Quatro tipos de operações OLAP são utilizadas para análise de dados. Operações *drill* navegam nos dados modificando o nível de granularidade da

consulta, enquanto que operações de *slice and dice* navegam nas dimensões utilizadas pelo usuário final.

Os tipos de operações são explicados a seguir.

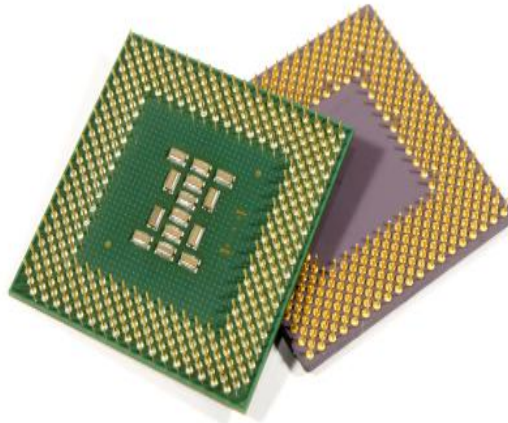
- *Drill down* e *roll up*: o *drill down* ocorre quando se aumenta o nível de detalhe da informação, diminuindo seu nível de granularidade. O processo inverso é o *roll up*, quando se aumenta o nível de granularidade da informação, diminuindo seu nível de detalhe. Os caminhos de navegação são determinados pelas hierarquias de dimensão. Por exemplo, o usuário faz um *drill down* quando vê os dados disponibilizados por meses do ano e depois os vê por dias do mês. Quando faz o inverso, o usuário executa um *roll up*.
- *Drill across*: ocorre quando o usuário pula um nível intermediário de uma hierarquia dentro da dimensão. Por exemplo, detalhar diretamente um dado por ano para um dado por trimestre, sendo que existe o atributo mês entre estes atributos.
- *Drill through*: ocorre quando o usuário passa de uma informação contida numa dimensão para outra. Por exemplo, primeiro o usuário está analisando a informação pela dimensão tempo e depois passa a analisá-la pela dimensão região.
- *Slice and dice*: são operações para navegar pelos dados através de um cubo, que reduzem o escopo dos dados em análise. O *slice* separa parte do cubo, mas mantém a mesma perspectiva de visualização dos dados. Já o *dice* é a operação que muda a perspectiva de visualização. Um exemplo disto é quando o usuário visualiza o número de vendas de aparelhos de telefone por tipo de telefone, fixo ou celular, na região sudeste do Brasil. Fazendo uma operação *slice* o usuário poderia visualizar o número de vendas apenas de celulares na região sudeste. Já a operação *dice* seria trocar o sentido da análise. Se antes os dados eram vistos no sentido de tipo de telefone para região, passando por um *dice* seria visto no sentido de região para tipo de telefone. O *dice* seria como inverter as informações de coluna e linha de uma tabela.

Estrutura de Arquivos e de Banco de Dados

- Existem diversos tipos de armazenamento;
- São classificados por:
 - Velocidade de acesso aos dados;
 - Custo de mídia;
 - Confiabilidade;
- Tipos de armazenamento:
 - Cache;
 - Memória Principal;
 - Memória *Flash*;
 - Disco magnético;
 - Disco ótico;
 - Fita.

Cache

A figura abaixo ilustra uma memória cache:



- Características:
 - Mais rápidas, porém mais caras;
 - Pequeno espaço de armazenamento;
 - Gerenciado pelo Sistema Operacional.

Memória Principal

A figura abaixo é um exemplo de memória principal:



- Características:
 - Memória pequena para o armazenamento de um banco de dados;
 - Executa processos gerenciados pelo Sistema Operacional;
 - Caso haja perda de energia, os dados são perdidos.

Memória *Flash*

Abaixo, temos um exemplo de memórias do tipo *flash*:



- Características:
 - Dados não são perdidos caso haja falta de energia;

- Leitura de dados similar a da memória principal, porém com escrita mais demorada;
- Bastante utilizada para gerenciamento de pequenas quantidades de arquivos.

Disco Magnético

Alguns exemplos de discos magnéticos:



- Características
 - Armazena um Banco de Dados por completo;
 - Oferecem maior capacidade de armazenamento;
 - Disponibilizam o acesso dos dados para a memória volátil.

Discos Óticos

As imagens abaixo ilustram os discos óticos:



- Características:

- Disponibilizam uma maneira de armazenar os dados em um disco flexível;
- Utilizados para gravar arquivos de dados;
- Oferecem uma velocidade de acesso rápida comparada a outros dispositivos de armazenamento.

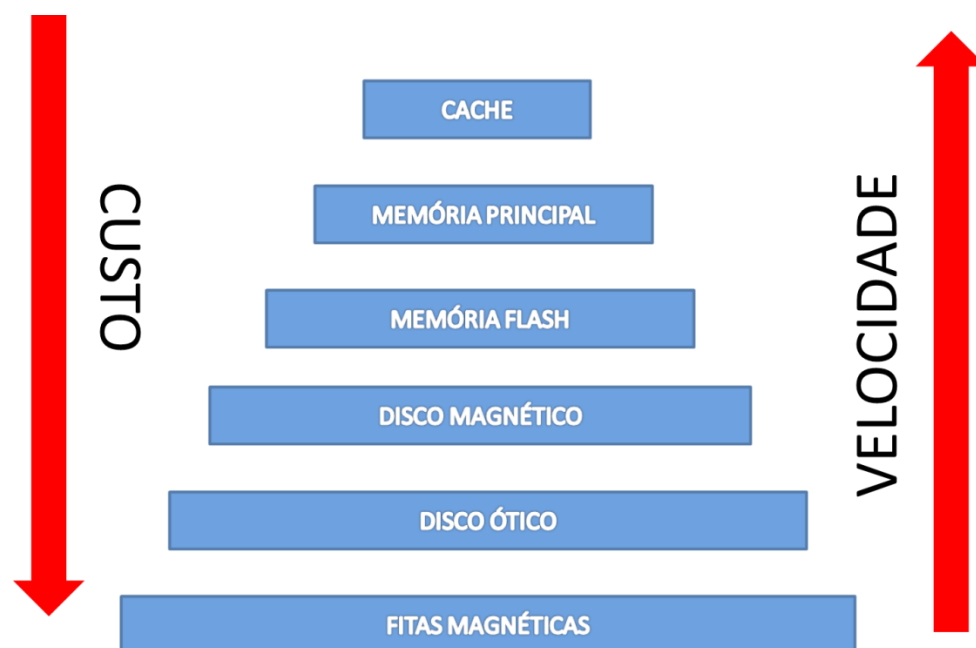
Fita

Alguns exemplos de fitas são apresetandos na imagem abaixo:



- Características:
 - Muito utilizado para backup ou arquivo de dados;
 - Acesso a dados mais lento;
 - Conhecido como memória de acesso sequencial.

Comparando os tipos de armazenamentos listados acima, podemos criar uma pirâmide de “Custo x Velocidade”. Quanto maior o custo, maior será a velocidade de acesso aos dados e vice-versa (próxima página).



Analisando a imagem, podemos ver que quem está no topo da velocidade consequentemente tendo um custo maior é a memória cache. De maneira inversa, com baixo custo mas com uma velocidade menor, temos as fitas magnéticas.

Raid (*Redundant Array of Independent Disks*)

RAID é a sigla para *Redundant Array of Independent Disks* ou, em tradução livre, algo como Matriz Redundante de Discos Independentes. Trata-se, basicamente, de uma solução computacional que combina vários discos rígidos (HDs) para formar uma única unidade lógica de armazenamento de dados.

E o que é unidade lógica? Em poucas palavras, no que se refere a RAID, trata-se de fazer com que o sistema operacional enxergue o conjunto de HDs como uma única unidade de armazenamento, independente da quantidade de dispositivos que estiver em uso. Hoje, além de HDs, é possível montar sistemas RAID baseados em SSD.

Fazer com que várias unidades de armazenamento trabalhem em conjunto resulta em muitas possibilidades:

- Se um HD sofrer danos, os dados existentes nele não serão perdidos, pois podem ser replicados em outra unidade (redundância);
- É possível aumentar a capacidade de armazenamento a qualquer momento com a adição de mais HDs;
- O acesso à informação pode se tornar mais rápido, pois os dados são distribuídos a todos os discos;
- Dependendo do caso, há maior tolerância a falhas, pois o sistema não é paralisado se uma unidade parar de funcionar;

Um sistema RAID pode ser mais barato que um dispositivo de armazenamento mais sofisticado e, ao mesmo tempo, oferecer praticamente os mesmos resultados.

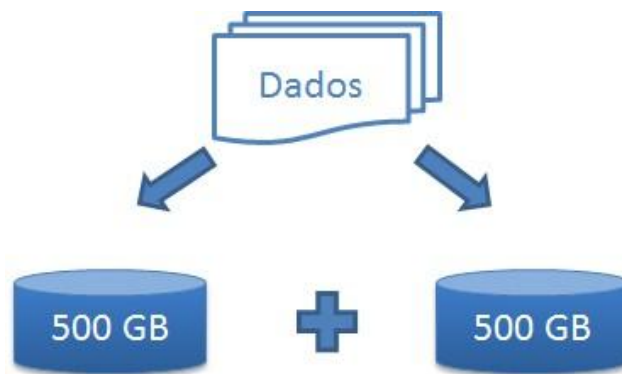
Níveis de RAID

Para que um sistema RAID seja criado, é necessário utilizar pelo menos dois HDs (ou SSD's). Mas não é só isso: é necessário também definir o nível de RAID do sistema. Cada nível possui características distintas justamente para atender às mais variadas necessidades. Iremos citar alguns níveis existentes.

RAID 0

Também conhecido como *striping* (fracionamento), o nível RAID 0 é aquele onde os dados são divididos em pequenos segmentos e distribuídos entre os discos. Trata-se de um nível que não oferece proteção contra falhas, já que nele não existe redundância. Isso significa que uma falha em qualquer um dos discos pode ocasionar perda de informações para o sistema todo, especialmente porque "pedaços" do mesmo arquivo podem ficar armazenados em discos diferentes.

O foco do RAID 0 acaba sendo o desempenho, uma vez que o sistema praticamente soma a velocidade de transmissão de dados de cada unidade. Assim, pelo menos teoricamente, quanto mais discos houver no sistema, maior é a sua taxa de transferência. Não é difícil entender o porquê: como os dados são divididos, cada parte de um arquivo é gravada em unidades diferentes ao mesmo tempo. Se este processo acontecesse apenas em um único HD, a gravação seria um pouco mais lenta, já que teria que ser feita sequencialmente.

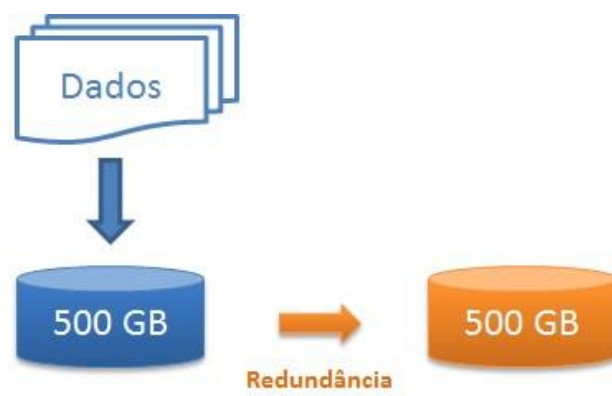


Por ter estas características, o RAID 0 é muito utilizado em aplicações que lidam com grandes volumes de dados e não podem apresentar lentidão, como tratamento de imagens e edição de vídeos.

RAID 1

O RAID 1 é, provavelmente, o modelo mais conhecido. Nele, uma unidade “duplica” a outra, isto é, faz uma “cópia” da primeira, razão pela qual o nível também é conhecido como *mirroring* (espelhamento). Com isso, se o disco principal falhar, os dados podem ser recuperados imediatamente porque existe cópias no outro.

Perceba que, por conta desta característica, sistemas RAID 1 devem funcionar em pares, de forma que uma unidade sempre tenha um “clone”. Na prática, isso significa que um sistema RAID composto por dois HDs com 500 GB cada terá justamente esta capacidade, em vez de 1 TB.



O nível RAID 1 é claramente focado na proteção dos dados, ou seja, não torna o acesso mais rápido. Na verdade, pode até ocorrer uma ligeira perda de desempenho, uma vez que o processo de gravação acaba tendo que acontecer duas vezes, uma em cada unidade.

É importante observar, no entanto, que o uso de RAID 1 não dispensa soluções de backup. Como a duplicação dos dados é feita praticamente em tempo real, significa que se uma informação indevida for gravada na primeira unidade (como um vírus) ou se um arquivo importante for apagado por engano, o mesmo acontecerá no segundo disco. Por isso, RAID 1 se mostra mais adequado para proteger o sistema de falhas “físicas” das unidades.

RAID 2, 3 e 4

Os níveis de RAID mostrados até agora são os mais utilizados, mas há alguns menos conhecidos, entre eles, RAID 2, RAID 3 e RAID 4:

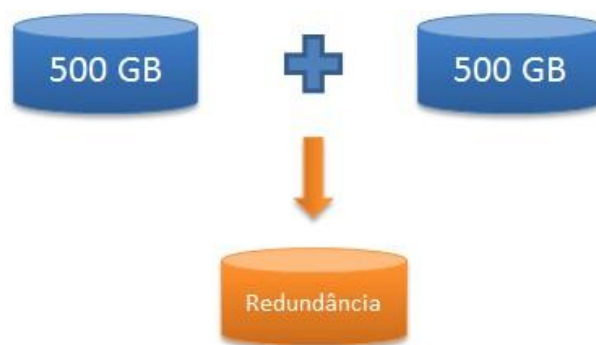
RAID 2

RAID é um tipo de solução de armazenamento que surgiu no final dos anos 1980. Naquela época e nos anos seguintes, os HDs não tinham o mesmo padrão de confiabilidade que têm hoje. Por este motivo, foi criado o RAID 2.

Ele é, até certo ponto, parecido com o RAID 0, mas conta com um mecanismo de detecção de falhas do tipo ECC (*Error Correcting Code*). Hoje, este nível quase não é mais utilizado, uma vez que praticamente todos os HDs contam com o referido recurso.

RAID 3

Este é um nível parecido com o RAID 5 por utilizar paridade. A principal diferença é que o RAID 3 reserva uma unidade de armazenamento apenas para guardar as informações de paridade, razão pela qual são necessários pelo menos três discos para montar o sistema. Este nível também pode apresentar maior complexidade de implementação pelo fato de as operações de escrita e leitura de dados considerarem todos os discos em vez de tratá-los individualmente.



RAID 4

O RAID 4 também utiliza o esquema de paridade, tendo funcionamento similar ao RAID 3, com o diferencial de dividir os dados em blocos maiores e de oferecer acesso individual a cada disco do sistema.

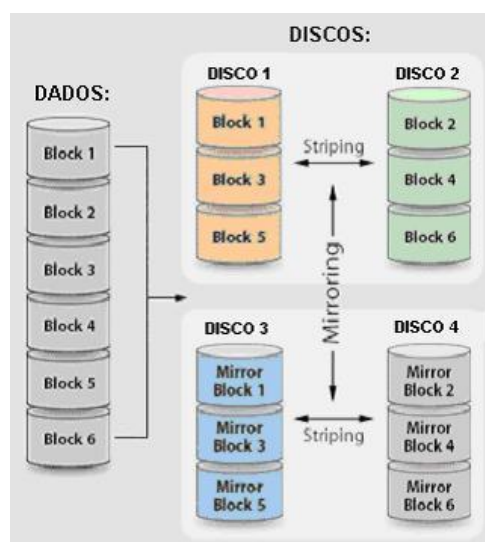
Este nível pode apresentar algum comprometimento de desempenho, pois toda e qualquer operação de gravação exige atualização na unidade de paridade. Por este motivo, seu uso é mais indicado em sistemas que priorizam a leitura de dados, ou seja, que realizam muito mais consultas do que gravação.

RAIDS Híbridos

Alguns RAID's são resultados da utilização de mais de um tipo de RAID em conjunto, como veremos abaixo.

RAID 01 ou RAID 0 + 1

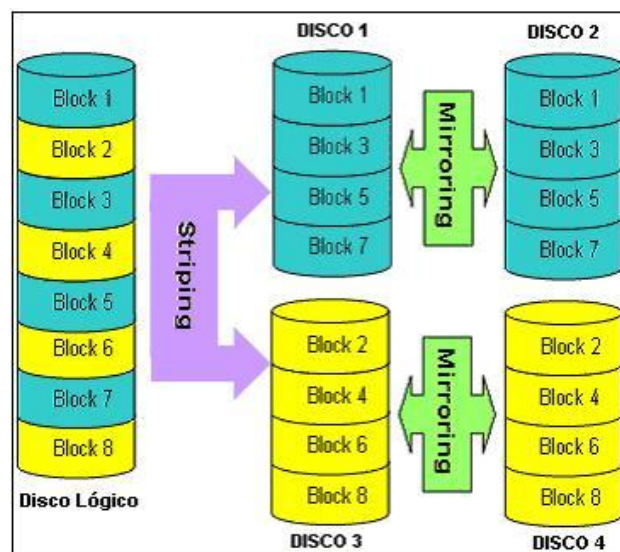
Em uma implementação RAID 0+1, os dados são espelhados através de grupos de discos segmentados, isto é, os dados são primeiro segmentados e para cada segmento criados é feito um espelho, como demonstrado na figura abaixo:



Na figura acima vemos que os discos 1 e 2 formam um RAID 0 sendo após espelhados pelo discos 3 e 4 também em RAID 0, formando assim RAID 1 sobre RAID 0. Apesar de ser uma configuração que proporciona alta performance, se perdermos um disco em um dos lados, praticamente teremos uma configuração em RAID 0, porque em uma configuração RAID 0 se um disco falha todo o conjunto falhará. Neste caso, se o disco 1 falhar, então o disco 2 que está intacto ficará inutilizado, restando assim os discos 3 e 4 em RAID 0.

RAID 10 ou RAID 1 + 0

Em uma implementação RAID 1+0, os dados são segmentados através de grupos de discos espelhados, isto é, os dados são primeiro espelhados e para depois serem segmentados como demonstrado na figura abaixo:



Na figura acima vemos que os discos 1 e 2 formam um RAID 1 e os discos 3 e 4 também sendo após segmentados em RAID 0, formando assim RAID 0 sobre RAID 1. Além de ser uma configuração que proporciona o mesmo nível de performance proporcionado pelo RAID 01, o RAID 10 proporciona mais tolerância à falhas que o RAID 01 porque poderíamos ter uma falha simultânea dos discos 1 e 3 e ainda assim o conjunto estaria intacto, pois teríamos os espelhos em perfeito funcionamento. No meu ponto de vista, este conjunto é o mais indicado nos casos onde necessitamos aliar performance e redundância, como é o caso, por exemplo, de bancos de dados Oracle de alta performance.

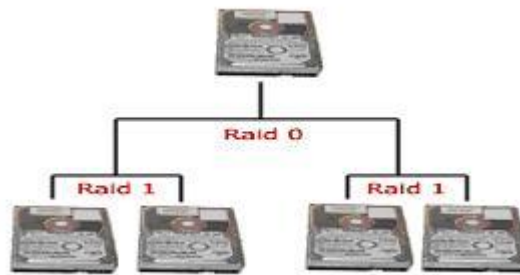
Conclusão sobre RAID 01 e RAID 10

Nos dois casos (0+1 ou 1+0), a perda de um único disco não resultará na falha do sistema RAID. A diferença aparece no caso da perda de um segundo disco que dependendo do disco, o sistema RAID 0+1 ficaria em desvantagem sobre o sistema RAID 1+0. Uma outra diferença é na velocidade de recuperação, porque caso ocorra uma falha de disco, no sistema RAID 1+0 será necessário apenas re-espelhar um disco, ao contrário do sistema RAID 0+1 que será necessário espelhar todo um conjunto segmentado. Portanto não se esqueça que RAID 01 é diferente de RAID 10.

RAID 01



RAID 10



Técnicas de Recuperação de Banco de Dados

Este tópico tem como objetivo realizar um breve levantamento sobre a importância da recuperação de Banco de Dados.

- Visão Geral:
 - Importante para recuperação de Banco de Dados;
 - Utilização de transações nas operações que armazenam os dados;
 - Garante que os dados não serão perdidos caso ocorra alguma falha;
 - Armazenamento no arquivo Log do BD.
- Existem basicamente três tipos de recuperação de Banco de Dados:
 - *Simple*;

- *Bulk-Logged*;
- *Full*.

Simple

- Log de transações é pouco utilizado nesse modelo de recuperação;
- Restaura informações somente do último backup;
- Utilizado em bancos de teste ou em bancos que não exista problema a se perder os dados.

Bulk-Logged

- Registra mais informações que o modelo *Simple*;
- Não registra operações de volume: SELECT INTO, INSERT, CREATE INDEX e operações com campos do tipo texto;
- Nessa opção nem todos os dados são inseridos no Log do Banco de Dados.

Full

- Realiza backup do Log de transações;
- Armazena todas as informações em um Log;
- Recupera o Banco de Dados até o último minuto em que ocorreu o problema;
- Oferece um nível maior de proteção;
- Indicado usar este sempre que possível.

Segurança de Banco de Dados

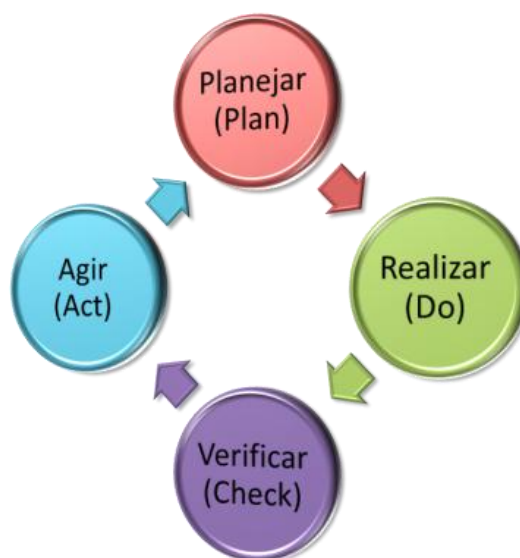
Este tópico será um breve comentário da importância da segurança dos Bancos de Dados, que é o repositório do bem mais valioso das Organizações dos dias de hoje: a informação.

Se baseia basicamente em quatro princípios:

- **Confidencialidade:** capacidade de um sistema permitir que um usuário acesse determinadas informações ao mesmo tempo que impede que usuários não autorizados as vejam;
- **Integridade:** garante que a informação manipulada mantenha todas as características originais estabelecidas pelo proprietário das informações, como o controle de mudanças e garantia do seu ciclo de vida (nascimento, manutenção e destruição);
- **Disponibilidade:** garante que a informação esteja sempre disponível para o uso legítimo, ou seja, para usuários autorizados pelo proprietário da informação;
- **Irretratabilidade:** garante a impossibilidade de negar a autoria em relação a uma informação.

A Política de Segurança da Informação segue o modelo PDCA:

- **Planejar (*Plan*):** identificar os ativos a serem protegidos através da análise de riscos;
- **Realizar (*Do*):** colocar em ação o planejamento realizado e aplicar os mecanismos necessários para atender os requisitos de proteção identificados;
- **Verificar (*Check*):** monitorar se o plano está sendo seguido e se necessita ser revisado;
- **Agir (*Act*):** manter um plano de ação para reiniciar o ciclo e planejar mudanças na política estabelecida pelo plano anterior.



Segurança Física

- Manter o servidor em uma sala onde somente pessoas autorizadas tem acesso;
- Manter sistemas de energia alternativos (*no-breaks*, geradores);
- Manter sistemas de refrigeração;
- Manter backups distribuídos, ou seja, que não fiquem no mesmo lugar que o servidor e, de preferência, em mais de um lugar;
- Organizar o meio físico, como computadores, servidores, identificação de cabos, etc.

Segurança Via Serviços

- Instalar somente serviços do SGBD que são utilizados;
- De acordo com a necessidade, ir instalando os demais serviços disponíveis;
- No script abaixo, é possível visualizar os serviços instalados e ativar ou desativar um serviço. No exemplo, o serviço “Remote Access” está sendo ativado:

```
sp_configure 'show advanced options', 1;
```

```
GO
```

```
RECONFIGURE;
```

```
GO
```

```
sp_configure 'remote access', 1;
```

```
GO
```

```
RECONFIGURE;
```

```
GO
```

Contas de Serviços

- Configuração de níveis de acessos aos serviços;
- Cada serviço acessa somente o que precisa;

- Tipos de contas (SQL Server 2008):
 - Domain user que não é administrador do Windows;
 - Local user que não é administrador do Windows;
 - Network Service;
 - Local System;
 - Local user e administrador Windows;
 - Domain user e administrador Windows.

Outras Ferramentas

- Ferramenta de análise de segurança;
- Atualização sempre que possível do Sistema Operacional do do Sistema Gerenciador de Banco de Dados (SGBD): *Patching SQL Server e Windows*;
- TDE (*Transparent Data Encryption*):
 - Criptografia do Banco de Dados.

Auditoria no SQL Server

- Utilizado para obter informações gerenciais do que ocorre no sistema;
- Ferramentas:
 - *SQL Server Audit*:
 - Geração de Logs:
 - A nível de servidor;
 - A nível de Banco de Dados.
 - *Change Data Capture* (CDC):
 - Logs de INSERT, UPDATE, DELETE;
 - Controle de permissões de alterações com o *rollback*.

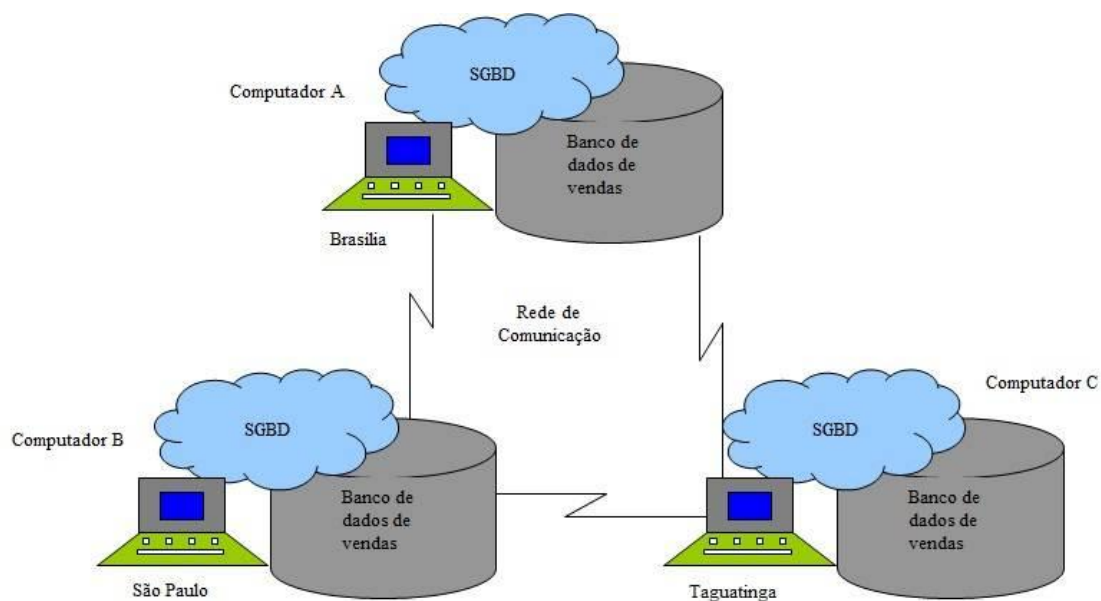
Autenticação

- Existem dois tipos:
 - *Windows Authentication*: recomendável utilizar sempre que possível;
 - *Mixed Mode Authentication*: utilizar regras de senhas complexas.

Banco de Dados Distribuídos

Este tópico tem como objetivo dar uma breve descrição sobre Sistemas de Banco de Dados Distribuídos.

- Características:
 - Relação entre diversos computadores (nós);
 - Gerenciamento das transações é feito em cada nó;
 - Comunicação por ser feita com o uso de:
 - Redes de alta velocidade;
 - Redes sem fio.



- Características do armazenamento:
 - Replicação:
 - Réplica dos dados (espelhamento);
 - Entre dois ou mais nós.
 - Fragmentação:
 - Divisão dos dados em fragmentos (quebra dos dados);
 - Fragmentação e Replicação:
 - Divisão em fragmentos e replicação dos mesmos.

Transações

- Atomicidade: todas as operações serão refletidas no Banco de Dados ou nenhuma será;
- Consistência: execução de uma transação preservará a consistência dos dados;
- Isolamento: transação não toma conhecimento das transações concorrentes;
- Durabilidade: transação efetuada com sucesso deve ser persistida no Banco de Dados.

Vantagens

- Compartilhamento e controle distribuído;
- Autonomia;
- Crescimento incremental;
- Disponibilidade;
- Consultas em paralelo.

Desvantagens

- Custo elevado;
- Grande potencial no aumento de *bugs*;

- Necessidade de coordenação entre os nós;
- Perda de dados durante a comunicação entre os nós.

Banco de Dados Orientados a Objeto

Hoje, o banco de dados orientados a objeto é um fator emergente que integra banco de dados e a tecnologia de orientação a objetos. Por um lado, a necessidade de realizar manipulações complexas para os banco de dados existentes e uma nova geração de aplicações de banco de dados geralmente requisitam mais diretamente um banco de dados orientado a objeto. Por outro lado, aplicações de linguagens orientadas a objeto e sistemas estão exigindo capacidades de banco de dados, tais como continuidade, simultaneidade e transações, dos seus ambientes. Estas necessidades estão levando à criação de sistemas poderosos, chamados banco de dados orientados a objeto.

Os bancos de dados orientados a objeto iniciaram-se primeiramente em projetos de pesquisa nas universidades e centros de pesquisa. Em meados dos anos 80, eles começaram a se tornar produtos comercialmente viáveis. Hoje, eles são mais de 25 produtos no mercado.

Conceitos Básicos

O desenvolvimento dos Sistemas de Gerenciamento de Banco de Dados Orientado a Objetos (SGBDOO) teve origem na combinação de ideias dos modelos de dados tradicionais e de linguagens de programação orientada a objetos.

No SGBDOO, a noção de objeto é usada no nível lógico e possui características não encontradas nas linguagens de programação tradicionais, como operadores de manipulação de estruturas, gerenciamento de armazenamento, tratamento de integridade e persistência dos dados.

Os modelos de dados Orientados a Objetos tem um papel importante nos SGBD's porque, em primeiro lugar, são mais adequados para o tratamento de objetos complexos (textos, gráficos, imagens) e dinâmicos (programas, simulações). Depois, por possuírem maior naturalidade conceitual e, finalmente, por estarem em consonância com fortes tendências em linguagens de programação e engenharia de software. O casamento entre as linguagens de programação e banco de dados é um dos problemas que estão sendo tratados de forma mais adequada no contexto de Orientação a Objetos.

Apresenta-se adiante os conceitos básicos de modelos de dados e SGBD's Orientados a Objetos.

Modelos de Dados Orientados a Objetos

Superficialmente, pode-se dizer que Orientação a Objetos corresponde à organização de sistemas como uma coleção de objetos que integram estruturas de dados e comportamento. Além desta noção básica, a abordagem inclui um certo número de conceitos, princípios e mecanismos que a diferenciam das demais. Seus principais conceitos são apresentados em seguida.

Abstração

É a consideração apenas das propriedades comuns de um conjunto de objetos, omitindo os detalhes, utilizada com frequência na definição de valores similares e na formação de um tipo a partir de outro, em diferentes níveis de abstração. O uso de abstrações permite a geração de tipos baseada em hierarquias de tipos e de relacionamentos.

Os principais conceitos de abstração utilizados em banco de dados são generalização e agregação. A generalização corresponde à associação “é um”

onde, a partir de propriedades comuns de diferentes entidades, é criada outra entidade. O processo inverso é a especialização. A agregação corresponde a associação “parte de”.

Objeto

Os objetos são abstrações de dados do mundo real, com uma interface de nomes de operações e um estado local que permanece oculto. As abstrações da representação e das operações são ambas suportadas no modelo de dados orientado a objetos, ou seja, são incorporadas as noções de estruturas de dados e de comportamento. Um objeto tem um estado interno descrito por atributos que podem apenas ser acessados ou modificados através de operações definidas pelo criador do objeto. Um objeto individual é chamado de instância ou ocorrência de objeto. A parte estrutural de um objeto (em banco de dados) é similar à noção de entidade no modelo Entidade-Relacionamento.

Identidade de Objeto

Num modelo com identidade de objetos, estes têm existência independente de seus valores correntes e dos endereços de armazenamento físico. A identidade do objeto é geralmente gerada pelo sistema. A impossibilidade de garantir a identificação de objetos exclusivamente através de suas propriedades estruturais e comportamentais motivou a definição de identificadores únicos de objetos, que persistem no tempo de forma independente ao estado interno do objeto.

A identidade de objetos elimina as anomalias de atualização e de integridade referencial, uma vez que a atualização de um objeto será automaticamente refletida nos objetos que o referenciam e que o identificador de um objeto não tem seu valor alterado.

Objetos Complexos

Os objetos complexos são formados por construtores (conjuntos, listas, tuplas, registros, coleções, *arrays*) aplicados a objetos simples (inteiros, booleanos, *strings*). Nos modelos Orientados a Objetos, os construtores são em geral ortogonais, isto é, qualquer construtor pode ser aplicado a qualquer objeto. No modelo relacional este não é o caso, visto que só é possível aplicar o construtor de conjuntos às tuplas e o construtor de registro a valores atômicos.

A manutenção de objetos complexos, independente de sua composição, requer a definição de operadores apropriados para sua manipulação como um todo, e transitivos para seus componentes. Exemplos destas operações são: a atualização ou remoção de um objeto e cópia profunda ou rasa.

Encapsulamento

O encapsulamento possibilita a distinção entre a especificação e a implementação das operações de um objeto, além de prover a modularidade que permite uma melhor estruturação das aplicações ditas complexas, bem como a segurança dentro do sistema. Em Banco de Dados se diz que um objeto está encapsulado quando o estado é oculto ao usuário e o objeto pode ser consultado e modificado exclusivamente por meio das operações a ele associadas.

Existe uma certa discussão sobre as consultas em Banco de Dados quando está incorporada a noção de encapsulamento: Deve-se tornar visível apenas as operações e deixar ocultos os dados e as implementações ? É interessante relaxar o encapsulamento apenas para as consultas? Como deve ser realizada a otimização de consultas em SGBDOO com encapsulamentos?

Tipo de Objetos

O tipo de objeto pode ser visto como a descrição ou especificação de objetos. Um tipo possui duas partes, interface (visível para o usuário do tipo) e implementação (visível só para o usuário construtor do tipo).

Existem várias vantagens em se ter um sistema de tipos em um modelo de dados. Além de modularidade e segurança, do ponto de vista da evolução do sistema os tipos são especificações do comportamento que podem ser compostos e modificados incrementalmente, para formar novas especificações.

Classes

Um conjunto de objetos que possui o mesmo tipo (atributos, relacionamentos, operações) pode ser agrupado para formar uma classe. A noção de classe é associada ao tempo de execução, podendo ser vista como uma representação por extensão, enquanto que o tipo é uma representação intencional. Cada classe tem um tipo associado, o qual especifica a estrutura e o comportamento de seus objetos. Assim, a extensão da classe denota o conjunto dos objetos atualmente existentes na classe e o tipo provê a estrutura destes objetos.

Herança

Herança é um mecanismo que permite ao usuário definir tipos de forma incremental, por refinamento de outros já existentes, permitindo composição de tipos em que as propriedades de um ou mais tipos são reutilizadas na definição de um novo tipo. De fato, ela corresponde a transferência de propriedades estruturais e de comportamento de uma classe para suas subclasses.

As principais vantagens de herança são prover uma maior expressividade na modelagem dos dados, facilitar a reusabilidade de objetos e definir classes por refinamento, podendo fatorar especificações e implementações como na adaptação de métodos gerais para casos particulares, redefinindo-os para estes, e simplificando a evolução e a reusabilidade de esquemas de banco de dados.

Tipos de Herança

Os dois tipos de herança, simples e múltipla, são descritos a seguir:

- **Herança Simples:** Na herança simples, um certo tipo pode ter apenas um supertipo, da mesma forma uma subclasse só herda diretamente de uma única classe. Podemos classificar esta herança em quatro subtipos: de substituição, de inclusão, de restrição e de especialização.
- **Herança Múltipla:** Nesta herança um tipo pode ter supertipos e os mesmos refinamentos de herança simples. Há basicamente dois tipos de conflitos referentes à herança múltipla: entre o tipo e o supertipo e entre múltiplos supertipos. O primeiro pode ser resolvido dando-se prioridade à definição presente no tipo, e não a no supertipo. Com os conflitos entre múltiplos supertipos, como uma resolução por default pode causar heranças não desejadas, a abordagem mais segura é baseada na requisição explícita da intervenção do usuário.

Métodos e Mensagens

Um método, em relação a um objeto, corresponde ao comportamento dos objetos, implementando uma operação associada a uma ou mais classes, de forma similar aos códigos dos procedimentos usados em linguagens de programação tradicionais, que manipula o objeto ou parte deste. Cada objeto tem um certo número de operações para ele definida. Para cada operação pode-se ter um ou mais métodos de implementação associados.

As mensagens são a forma mais usada para se ativar os métodos. Num SGBDOO os objetos se comunicam e são ativados através de mensagens enviadas entre eles.

Polimorfismo

Em sistemas polimórficos uma mesma operação pode se comportar de diferentes formas em classes distintas. Como exemplo temos a operação *print* que será implementada de forma diferente se o objeto correspondente for um texto ou uma imagem: dependendo do objeto teremos um tipo de impressão. Tem-se também polimorfismo quando ocorre a passagem de diferentes tipos de objetos como parâmetros enviados a outros objetos.

Um mesmo nome pode ser usado por mais de uma operação definida sobre diferentes objetos, o que caracteriza uma sobrecarga (*overloading*). A redefinição do operador para cada um dos tipos de objetos definidos caracteriza uma sobreposição (*overriding*). As operações são ligadas aos programas em tempo de execução caracterizando o acoplamento tardio ou *late binding*.

Outros conceitos

Finalmente há duas propriedades fundamentais para a construção de um SGBDOO: extensibilidade e completude computacional. A primeira garante que o conjunto de tipos oferecidos pelo sistema permite a definição de novos tipos e não há distinção entre os tipos pré-definidos e os definidos pelo usuário. A segunda implica que a linguagem de manipulação de um Banco de Dados Orientado a Objetos pode exprimir qualquer função computacional.

Tipos de dados

TINYINT: Armazena valores numéricos inteiros, variando de 0 a 256.

SMALLINT: Armazena valores numéricos inteiros, variando de -32.768 a 32.767.

INT: Armazena valores numéricos inteiros, variando de -2.147.483.648 a 2.147.483.647.

BIGINT: Armazena valores numéricos inteiros, variando de -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807.

SMALLMONEY: Valores numéricos decimais variando de -214,748.3648 a 214,748.3647.

MONEY: Valores numéricos decimais variando de -922,337,203,685,477.5808 a +922,337,203,685,477.5807.

NUMERIC(18,0): Armazena valores numéricos com casas decimais, utilizando precisão. O primeiro número entre os parênteses representa a quantidade de inteiros a ser armazenado, o segundo número, indica a quantidade de casas decimais do número.

DECIMAL(18,0): Tem as mesmas funcionalidades do tipo NUMERIC, a diferença é que o DECIMAL faz parte do padrão ANSI e NUMERIC é mantido por compatibilidade.

FLOAT: Armazena valores numéricos aproximados com precisão de ponto flutuante, variando de $-1.79E + 308$ a $1.79E + 308$.

REAL: Armazena valores numéricos aproximados com precisão de ponto flutuante, variando de $-3.40E + 38$ a $3.40E + 38$.

Tipo BIT:

BIT: Armazena bits, ou seja, somente poderá conter os valores lógicos 0 ou 1.

Tipo data:

SMALLDATETIME: Armazena data e hora, com precisão de minutos.

DATETIME: Armazena data e hora, com precisão de centésimos de segundos.

TIME: Armazena somente hora. Pode armazenar segundos até a fração de 99999999.

DATE: Armazena somente data.

DATETIME2: É uma combinação dos tipos de dados DATE e TIME. A diferença para o tipo DATETIME é a precisão ao armazenar as horas.

DATETIMEOFFSET: Armazena valores data e hora com a combinação da hora do dia com o fuso horário. O intervalo de deslocamento do fuso horário é de -14:00 a +14:00.

Tipos caracteres:

CHAR(N): Armazena N caracteres fixos (até 8.000) no formato não Unicode. Independente da quantidade de caracteres utilizados, irá sempre armazenar o tamanho de caracteres do campo, sendo preenchido o restante com espaços em branco.

VARCHAR(N): Armazena N caracteres (até 8.000) no formato não Unicode.

VARCHAR(MAX): Armazena caracteres no formato não Unicode. MAX indica que o máximo a ser armazenado pode chegar a $2^{31}-1$ bytes.

TEXT: Armazena caracteres no formato não Unicode. Esse tipo de dado suporta até 2.147.483.647 caracteres e existem funções específicas para trabalhar com esse tipo de dado.

NCHAR(N): Armazena N caracteres fixos (até 4.000) no formato Unicode. Independente da quantidade de caracteres utilizados, irá sempre armazenar

o tamanho de caracteres do campo, sendo preenchido o restante com espaços em branco.

NVARCHAR(N): Armazena N caracteres (até 4.000) no formato Unicode.

NVARCHAR(MAX): Armazena caracteres no formato Unicode. MAX indica que o máximo a ser armazenado pode chegar a $2^{31}-1$ bytes.

NTEXT: Armazena caracteres no formato Unicode. Esse tipo de dado suporta até 1.073.741.823 caracteres e existem funções específicas para trabalhar com esse tipo de dado.

Outros tipos de dados:

BINARY(N): Armazena dados no formato binário, podendo chegar até 8.000 bytes. Independente da quantidade de dados armazenados será preenchido com espaços em brancos até completar o tamanho do campo.

VARBINARY(N): Armazena dados no formato binário, podendo chegar até 8.000 bytes.

VARBINARY(MAX): Armazena dados no formato binário, podendo chegar até $2^{31}-1$ bytes.

IMAGE: Armazena dados no formato binário, podendo chegar até 2,147,483,647 bytes.

SQL_VARIANT: Armazena todos os tipos de dados em um mesmo campo de uma tabela, com exceção dos tipos TEXT, NTEXT, TIMESTAMP e SQL_VARIANT.

TIMESTAMP: Este tipo de dados permite a geração automática de um valor binário para um campo de uma tabela.

UNIQUEIDENTIFIER: Esse tipo de dados é utilizado para a criação de um identificador global e único para uma tabela do SQL Server.

GEOMETRY: Armazena dados espaciais utilizando representação plana da Terra (Flat Earth).

GEOGRAPHY: Armazena dados espaciais utilizando representação redonda da Terra (Round Earth).

HIERARCHYID: É usado para representar uma posição em uma hierarquia. Uma coluna desse tipo não representa automaticamente uma árvore. É até a aplicação para gerar e atribuir valores hierarchyid de tal forma que a relação desejada entre as linhas é refletida nos valores.

XML: Armazena dados no formato XML, não podendo exceder a 2GB.

Conclusão

Espero que ao final dessa apostila, você tenha aprimorado e/ou aprendido coisas novas.

Este documento estará em constante atualização e sempre aberto à críticas e sugestões.

Um abraço e muito obrigado!

Referências

ALECRIM, Emerson, INFOWESTER. **Sistemas RAID (*Redundant Array of Independet Disks*)**. Jan. 2012. Disponível em: <<http://www.infowester.com/raid.php>>.

BLOG DA TI. **Tipos de Dados – SQL Server 2008**. Disponível em: <<http://www.blogdati.com.br/index.php/2010/03/tipos-de-dados-sql-server-2008/>>.

DEVMEDIA. **Boas Práticas de segurança para SQL Server 2008/2008 R2**. Ago. 2011.

DEVMEDIA. **Definindo Estratégia de Modelo de Recuperação para Banco de Dados SQL Server 2005**. Set. 2007.

DEVMEDIA. **Entendendo e Utilizando Índices na Otimização de Queries no SQL Server**. Jan. 2008.

DEVMEDIA. **Segurança em Banco de Dados – Aplicando Normas e Procedimentos**. Set. 2012.

DEVMEDIA. **Microsoft SQL Server 2005 – Trabalhando com Índices**. Mar. 2008.

DEVMEDIA. **Índices no SQL Server**. Out. 2010.

GONTIJO, Rafael Igor Freire. **Desenvolvimento do Data Mart Piloto do Centro Universitário de Patos de Minas**. Out. 2009.

JÚNIOR, Fernando Corrêa de Mello. **Sistemas de Banco de Dados II**. Ago. 2009.

LEGATTI, Eduardo, ORACLE BLOG. **Descomplicando RAID 01 (0+1) e RAID 10 (1+0)**. Mar. 2008. Disponível em: <<http://eduardolegatti.blogspot.com.br/2008/03/descomplicando-raid-01-01-e-raid-10-10.html>>.

MACHADO, Felipe Nery Rodrigues. **Tecnologia e projeto de Data Warehouse: uma visão multidimensional**. 2. ed. São Paulo: Érica, 2006. 318 p.

PORTAL EASYINFO. **Notícias Web**. Abr. 2010. Disponível em: <<http://migre.me/c6vgo>>.